ROBIN code review

Date: 2026-10-16
Code reviewed: `main` at `493f53f` (LooseLab/ROBIN, forked to gmc-norr/ROBIN); the independent review's findings were checked at `5be9e04`.
Scope: all of `src/robin` (91 Python files, about 82,000 lines), plus packaging, resources and CI config.
Prepared with: Claude Code (an AI-assisted review), plus a correction pass by Codex and an independent review by GPT 6.1 Sol. Treat each finding as a lead to check, not as a final verdict.
Updated: 2026-10-16. The independent review's 11 findings are merged in; each keeps its original number (IR-01 to IR-11) next to its new ID. The CNV-1, GUI-1 and SEC-1 fix sizes are corrected, the status lines are up to date, and the reproduction scripts are added.
Updated: 2026-10-30. SEC-9 is added. Fixes made in our fork are listed under "This fork and the original ROBIN", and each fixed finding has a **Status** line. Batch 2 (VAR-9, VAR-8, VAR-10) is fixed too.
Updated: 2026-10-30. Batch 3, step 3's quick wins, is fixed in our fork: GUI-1, REP-1, JOB-6, GUI-4, CNV-5, GUI-2, VAR-3 and VAR-7. JOB-6 is only partly fixed — the timeout constant and its configurability, not the two consequences that overlap JOB-7.
Updated: 2026-10-31. Two follow-ups from batch 3's own review are fixed in our fork: `robin add-panel` reads BED files wider than 6 columns (BED12 was saved garbled), and the variant panel's "Hide Results" keeps pointing at the file it showed.
Updated: 2026-10-31. Five findings from a review of the fix series (`reviews/2026-10-31-fix-series-review.md`) are fixed in our fork: stale CNV breakpoints after a flat run, an older LGA results file hiding a newer one, an LGA status timer that never stopped, SNV sites with no usable bases read as "None Detected", and `{file}` quoting inside `$(...)`. ClinVar sites no read covers now need review too, and that batch's own review led to two more fixes (a failed CNV detection, a "(legacy location)" caption). Six more bugs found along the way are fixed too; see the fork table.
Updated: 2026-10-31. Step 4 is fixed in our fork: CNV-1 (the control profile comes from the control's own counts, which also ends a 16-bit wrap found while fixing it), with CNV-3 (a batch's last BAM counted twice) in the same code.
Updated: 2026-11-01. Step 5, the methylation maths, is fixed in our fork except METH-7 (left for later): METH-1 to METH-6, METH-13 and METH-14, checked against `modkit pileup --combine-strands` on synthetic reads, and part of METH-12.
Updated: 2026-11-01. Step 6, target input and coverage, is fixed in our fork: TGT-1 to TGT-5, checked against samtools run once on all of a sample's reads (split into 1, 2 and 4 BAMs, in five ways of running, two workers at once among them). Per-base panel depth (D4) is noted as later work under "Suggested order of work". Its final review's fixes keep IGV's copy of `target.bam` in step with it, and stop a finalize from holding up the coverage step.
Updated: 2026-11-01. Step 7, making failures visible, is fixed in our fork: ERR-1 (all five layers), METH-8, VAR-4 and REP-6, and the reporting half of METH-9. Each failure was injected on purpose and checked where a user looks: the failed count, `failed_jobs.csv`, the GUI's errors and the PDF. Runs that looked clean can now show failed jobs, and a setup without R shows an MGMT failure on every run. Two reviews of step 7 found more; that is fixed too, with JOB-3 from step 8.
Updated: 2026-11-01. VAR-1 and VAR-2 are fixed in our fork, so step 9 is done. They were checked against ClinVar's own GRCh38 file. VAR-2 is not fixed as suggested below: matching CLNSIG alone would have dropped BRAF V600E and TERT C228T, whose germline label is "conflicting". A record now counts when ClinVar's germline, oncogenicity or somatic-impact label calls it pathogenic.
Updated: 2026-11-02. JOB-1 and JOB-5, from step 8, are fixed in our fork: no request for a classifier or SNP run is thrown away. They were checked through a real Coordinator, with requests sent while a run was busy. Their external review found two more problems, fixed too: batched SNP requests kept an older request's settings, and a finished run's waiting request could wait for good on a full backlog. A new finding is noted under step 8: on the `high` preset the Coordinator answers nothing while a job runs.
Updated: 2026-11-02. JOB-7 and the rest of JOB-6 are fixed in our fork. A finalize request merges once, a sample finalised before still gets its later reads, and an SNP run merges new reads first. Earlier SNP calls are reused only

when they came from the same reads. An external review found four gaps in that, fixed too: reads a worker was still adding, merges that missed reads, part-failed Clair3 runs, and a `target.bam` rewritten without a new batch. Batch 6 had already fixed JOB-6's batch-BAM deletion, and its lock already stopped JOB-7's damage: two finalizes at once, slowed, gave 0 bad of 40 with the lock and 39 of 40 without.
Updated: 2026-11-02. CNV-2, from step 8, is fixed in our fork: no worker keeps its own copy of a sample's CNV counts, a sample's CNV runs take turns, and the counts are written whole or not at all. It was checked with stand-in Ray workers in separate processes, CNV-2's own example among them. Its external review found one more gap, fixed too: an old-format counts file that couldn't be moved was skipped, and its counts were lost.
Updated: 2026-11-02. JOB-2, JOB-4 and JOB-8 are fixed in our fork. JOB-4: the Ray watcher sends waiting jobs on a timer, retries a failed send, and sends before the Coordinator stops; half-written BAMs are checked again, and one that never completes is shown. A ledger in the work folder records each BAM taken in and each of its steps' starts and ends. After a restart, finished steps are skipped and unstarted ones run; ones cut short by the stop, or that failed, are reported as failed files at every start, not run again. The scan, the watcher and the GUI share one intake, which takes a BAM in once (known by its contents, so a copy too) and only once it is complete, which is part of JOB-4. `master.csv` counts a BAM once. The watcher's BAMs get the reference. An external review found five more problems, fixed too: two older ones counted a BAM's data twice (a step's batch list overwritten by another step's, and Ray re-running a step whose worker died), `master.csv` became unreadable past about 3,600 BAMs, and failures disappeared after a restart. A second review found three more, fixed too: totals lost with a deleted `master.csv`, failed files put on the wrong step, and a failed-file report lost when the Coordinator was briefly away.
Updated: 2026-11-02. Two items noted for later by batches 8 and 9 are fixed in our fork. The target step shows each BAM whose coverage it could not add as a failed file, where it used to say "success" (TGT-3's second open point). The variant check names the regions it could not check, and the job counts as failed, where those genes used to vanish from the results. Four more silent cases of the same kind, found on the way, are fixed too. An external review found two gaps, fixed too: a BAM left out by another job's add-up was not shown after a restart, and a set whose first file move failed was reported lost though still staged. TGT-3's other open point, a commit that is not all-or-nothing, is next.
Updated: 2026-11-03. TGT-3 is fixed in full in our fork: a batch is added to all of a sample's coverage files, and its batch BAM handed to finalize, together or not at all. The files are prepared beside the real ones with a list of the steps that put them in place, and a stop part-way is finished by the next add-up or finalize. It was checked by killing a commit at each of its steps, and again during the recovery, against samtools. An external check of the plan found three gaps, fixed before the code: file names not yet on disk when the list was, a missing batch BAM taken as installed, and two tests too loose to tell. A power loss is designed for (the order of the syncs is tested), not measured. An external review of the code found three more gaps, fixed too: a batch BAM that finalize found lost did not stop the SNP calls, a batch BAM that couldn't be written was not reported after a stop, and a BAM whose files were moved only in part was not named. A second review of the code found two more, fixed too: a claim folder another job kept was not listed by finalize, and one whose name had not reached the disk was not made again.

This fork and the original ROBIN

This review is of the original ROBIN (LooseLab/ROBIN at `493f53f`). Our fork, gmc-norr/ROBIN, now has fixes that the original does not, so the two have diverged. **Every finding below is still open in the original ROBIN** unless LooseLab fixes it themselves. A finding fixed in our fork has a **Status** line.

Change in gmc-norr/ROBIN	Finding	Commits	What users notice
Skip modkit files with no panel sites	The "Error processing file … ['Chromosome', 'Start']" log line (context under METH-9)	`d02add2`	That error no longer appears for those files
Traceback and safe diagnostics when a modkit file fails to merge	—	`5be9e04`	A longer log line, with no patient positions or values

Change in gmc-norr/ROBIN	Finding	Commits	What users notice
Ray dashboard off by default, and on 127.0.0.1 when turned on	SEC-1	`3623e93`	Turn it on with `--ray-dashboard`. `workflow_ray` takes `--ray-dashboard` instead of `--no-ray-dashboard`
Ray workers get their own empty NiceGUI storage folder	Found after this review	`3623e93`	The `storage-general.json.tmp` error is gone
GUI storage moves from `./.nicegui` to `~/.robin/nicegui`	Found after this review	`c51e65d`, `944dbee`	Everyone sees the disclaimer and logs in again once
MGMT runs R without a shell	SEC-8	`2d3577e`, `0873744`	Nothing
Watched file names reach command templates as data	SEC-9	`4f26cf0`, `75485ce`	Inside a nested `sh -c`, `eval` or `ssh` command, use `"$ROBIN_FILE"` instead of `{file}`
Exact pins: numpy 1.26.4, pandas 2.1.4, scikit-learn 1.8.0; Python 3.12 only	PKG-3, part of PKG-6	`43db1df`	Installs refuse Python 3.13 and later
Lightweight variant check uses only the sample's own BAM and BED, and records them	VAR-9	`b9ffef8`	A missing file fails with "... for this sample at "
The GUI hides lightweight variant results that don't record their BAM	VAR-9	`d0e1975`	Old results aren't shown; run the variant analysis again
Every BED interval of a gene is checked	VAR-10	`20ea771`	Variants in later exons now appear
A pathogenic variant counts only with 3 or more alternate reads	VAR-8	`aeb5339`, `e5527a3`	New status "Insufficient Evidence" (1–2 reads, an indel or multi-base change, or reads showing neither allele); fewer genes flagged
The classification panel follows the classifiers the run enabled	GUI-1	`50da100`	With a classifier disabled, the panel keeps updating instead of freezing on the first result
Random Forest is scored against its own confidence cut-offs	REP-1	`229247f`, `f50ad61`	The PDF no longer says "High" where the GUI says "Medium" for the same sample
The sample-completion idle timeout defaults to an hour, not 60 s	JOB-6 (first part; the rest below)	`c4592a0`, `b23ef4f`	Samples are no longer finalised mid-run. Set `ROBIN_COMPLETION_TIMEOUT_SECONDS` to change it; a run that relied on 60 s now waits an hour
The CNV card reports an absent gain/loss count instead of the batch counter	GUI-4	`e677991`	"Gained: Not available" in place of "Gained: 37"
The end of a chromosome's signal is a terminator, not a breakpoint	CNV-5	`53165ed`	About 24 fewer regions in the master BED per run
Refresh timers survive a reconnect	GUI-2	`7ff93b1`	An open page keeps updating after a Wi-Fi blip or laptop sleep; a page that really closed is torn down up to 60 s later
A ClinVar refresh rebuilds the tabix index	VAR-3	`8a37a7a`	Pathogenic annotations no longer vanish after a ClinVar update
`robin add-panel` keeps every region of a gene	VAR-7	`1c8e95f`	Later exons and tiles stay in the panel, so coverage and SNP calling see them
`robin add-panel` reads BED files wider than 6 columns, by their first 4	Found while fixing VAR-7	`5af0a47`	A BED12 panel is saved with its real regions. It used to be saved garbled, with "added successfully"; 7 or 8 columns failed

Change in gmc-norr/ROBIN	Finding	Commits	What users notice
The variant panel's "Hide Results" offers the file it showed	VAR-9 follow-up	`754741d`	After "Hide", "View" opens the trusted file again, not a hard-coded path
A CNV run with no breakpoints replaces the last run's breakpoints	CNV-5 follow-up	`4032f8b`	After a flat run, old breakpoints leave the CNV plot and the master BED
The newest trusted variant results file is the one shown	VAR-9 follow-up	`f3f29cd`	A fresh negative is no longer hidden by an older positive in the other results location
The variant panel's status timer stops when a run ends	VAR-9 follow-up	`6acc03d`	An opened results view stays open instead of resetting every half second
An SNV site with no usable bases needs review	VAR-8 follow-up	`b26c410`	Such a gene reads "insufficient evidence", not "None Detected"
A ClinVar site no read covers needs review	VAR-8 follow-up	`65ab945`	Such a gene reads "insufficient evidence", and its summary counts `sites_without_reads`; a site just before a BED region is no longer picked
A failed CNV breakpoint detection keeps the last good breakpoints	CNV-5 follow-up	`2ca41c7`	A warning in the log instead of a silent "no breakpoints"
Fresh variant results are never captioned "(legacy location)"	VAR-9 follow-up	`0964752`	In a watch-folder setup, a result just produced no longer reads as old
The CNV plot's absolute view draws breakpoint lines	Found after this review	`80207bd`	Dashed breakpoint lines appear in the absolute view too
The per-gene variant path piles up its first site	Found after this review	`e60306a`	Nothing in ROBIN calls it; scripts using `analyze_genes` see every site
`${file}` in a command template is left to the shell	SEC-9 follow-up	`49a8a79`	A shell variable named `file` works in templates
Variant results record their UTC offset	VAR-9 follow-up	`7689900`	The newest result is picked correctly across a clock change
Variant results and CNV arrays are written whole or not at all	Found after this review	`fa77eb7`	No half-read files in the GUI; a failed write keeps the last good file
`robin add-panel` says "No gene names found in column 4"	Found after this review	`edb76bd`	A clear error instead of `'chrom'`
Results written whole keep the usual file permissions	Found after this review	`5749e5d`	Group members can still read results in a shared output folder
A failed pileup reads "pileup failed"; the gene details count unread sites	VAR-8 follow-up	`c71527b`, `3fe7ba5`	The panel says why a gene reads "insufficient evidence"
`{file}` after `$` inside single quotes is the placeholder	SEC-9 follow-up	`e8af255`	`'${file}'` prints `$` and the path, as the shell would
`{file}` inside `$(...)` is quoted for that context	SEC-9 follow-up	`ab5151b`	Templates like `"$(dirname "{file}")/x.bai"` work on paths with spaces
The CNV control's profile comes from the control's own counts	CNV-1	`b682da0`, `6ab6928`, `a06c4a2`, `2681f8b`	Gains and losses no longer fade over a run, and a sample no longer inherits the previous sample's events. CNV results change for the same data; a contig the control lacks leaves `CNV3.npy`
The CNV control's profile no longer wraps at 16 bits	Found while fixing CNV-1	`6ab6928`	Early in a run (up to about 24,000 reads), the CNV plot is no longer off by several copies
A multi-BAM CNV batch counts each BAM once	CNV-3	`391e159`	The batch's last BAM no longer counts double for the rest of the run

Change in gmc-norr/ROBIN	Finding	Commits	What users notice
Methylation counts are summed per CpG across batches and both strands	METH-1, METH-2, METH-4, METH-5	`c50365a`	Methylation values and classifier results change; NanoDX sees minus-strand reads. A sample's old parquet is converted on its next merge
matkit counts each call as modkit does	METH-3, METH-13, METH-14	`d304ded`, `b70d6a5`	Unsure calls leave coverage; implicit-mode Cs count as unmethylated; supplementary alignments no longer count; MGMT's input changes too
The CpG panel is read as 0-based, and its cache can't go stale	METH-6, METH-5, part of METH-12	`dfd909f`	The base before each CpG is no longer pulled in
A test against modkit pins step 5	Step 5	`dfe9f29`, `1c6c65f`, `5bcd583`	Nothing
Target coverage adds up exactly across BAMs	TGT-3	`d6c87ff`, `d752ce6`	"Global coverage" is mean depth (it was covered bases over genome length); target depths shift a little (length end − start; deletions don't count); covered bases per chromosome are exact, or blank in a folder written before
One locked step adds a batch to the totals	TGT-4	`d6c87ff`, `888b5f0`	Two workers at once no longer lose a batch's coverage, and a finalize's merge can't make one give up
Every BAM's panel reads reach target.bam, and a second finalize keeps the first	TGT-1, found while fixing it	`d6c87ff`, `24ebbc8`, `888b5f0`	SNP calls and IGV see the reads the coverage counts (IGV's copy is rebuilt when target.bam gains reads). `target.bam` holds the records overlapping the panel (`samtools view -L`), no longer the off-panel pieces of panel reads
The panel always reaches target.bam's regions	TGT-2	`063398d`, `d6c87ff`	Panel reads are kept even when the master BED was written without the panel
Read counts count primary records	TGT-5	`7773e2f`	Mapped reads, bases and yield drop for split reads; the 50,000-read gate counts reads
A test against samtools pins step 6	Step 6	`7b31e53`, `dd802d4`, `ee2fdd8`	Nothing
The panel's depth counts each base once	TGT-3 known limit (batch 8)	`c584ad3`, `b3bb841`, `af84c12`, `88a7584`	"Targets Estimated Coverage", the enrichment and the quality label drop where panel rows overlap (AML: 3.6% of its length); rCNS2 unchanged; a folder gains `target_union_coverage.csv`
A job's own errors decide whether it failed	ERR-1 (layers 2, 5)	`9bfe410`, `c52d86c`	Runs that looked clean can show failed jobs: MGMT, CNV, target, fusion and the classifiers fail when they file an error, on both schedulers and with `--no-use-ray`. A failed `target` no longer starts `igv_bam`
Failures and worker warnings reach the GUI	ERR-1 (layers 3, 4)	`ee7e3dd`	The workflow monitor's error counts and "Recent errors" show failures; "BAM Alignment Warning" and "Skipped (too many reads)" appear when their job ends
A file that fails in the methylation merge is reported	ERR-1 (layer 1), part of METH-9	`34ceadd`	`failed_jobs.csv` gains rows of kind `file_failed`; the batch's other files still go on
Classifiers report their failures	METH-8	`0241a47`	A setup without R shows an MGMT failure on every run; NanoDX saves no row without scores and writes no `errordf.csv`

Change in gmc-norr/ROBIN	Finding	Commits	What users notice
The ClinVar step records whether it ran	VAR-4	`1fe4925`	The report says when ClinVar annotation was unavailable or not recorded, instead of "No pathogenic variants were identified."
The fusion report tells not run, unreadable and none found apart	REP-6	`c60f817`, `6aad4ba`, `11573aa`	Three different messages where there was one; the methods text gives the filters the code applies; empty fusion pickles are written when nothing passes
Step 7's final review: the fusion report per type, ClinVar "not recorded" with rows, GUI updates that can't be lost	REP-6, VAR-4, ERR-1 (layers 3, 4)	`2052fe3`, `4ab371b`	One unreadable fusion type is said beside the other's candidates; a lost failure still refreshes the GUI's error counts
A failing step runs once	JOB-3	`7a14955`	A step that raised no longer runs a second time without its work folder, reference or panel
The GUI's fusion panel and `--no-use-ray` fail-only submissions	REP-6, ERR-1 (layers 2, 5)	`7a14955`	The panel says "not run", "could not be read" or "none found" per type; with `--no-use-ray`, a fail-only BAM's classifier failures are expected, as with Ray
A failed fusion build stays visible until a rebuild works	REP-6	`7a14955`, `97b34ee`, `4ab371b`, `6a01732`	A `.failed` file beside the fusion pickle; earlier candidates are labelled as from an earlier run, in the summary and the body; the next build rebuilds the pickle from the CSV, or fails again if the CSV is gone
The GUI's error summary never says "No errors detected." beside a count	ERR-1 (layer 4)	`6a01732`	"N failures recorded; the details are no longer available." when the failures left the event log before they were shown
A fusion candidate file that can't be read fails the job	REP-6 (batch 7b)	`97dd97e`	The fusion job fails and names the file, instead of losing its reads and saying "none found"; staged files stay until read; parquet files are written to a temporary name and renamed
A fault-injection test pins step 7	Step 7	`55eee97`, `410ce1b`, `61ca768`, `b04d86e`, `293b58f`	Nothing
A record is pathogenic when ClinVar's germline, oncogenicity or somatic label says so	VAR-2	`502fd72`	"Conflicting" no longer counts (182,153 records); BRAF V600E and TERT C228T still do, now by their oncogenic and Tier I labels; the SNP table shows ONC and SCI; a row without a snpEff annotation stays in the table
Every ClinVar record at a position is checked	VAR-1	`502fd72`	IDH1 R132C, KRAS G12D and the other records that shared a position are checked; the variant check lists them all
A ClinVar record with no alternate allele is skipped, not fatal to its gene	Found by the external review of VAR-1	`b97a709`	PTEN, BRCA2 and BRCA1 results no longer vanish; the results count the 13 skipped records
The variant check's HTML export counts found variants only	VAR-8 follow-up, found by the same review	`b97a709`	Its status and counts match the JSON results (nothing in ROBIN calls it)
Each sample keeps its own waiting classifier run	JOB-1	`f77bc5d`	With several samples, each sample's last classification follows its newest data; samples take turns, so classifiers run more often
A repeat SNP request runs	JOB-5	`f77bc5d`, `382588b`	Running the SNP analysis again works; requests sent during a run merge into one more run, with the newest settings; SNP requests are no longer batched

Change in gmc-norr/ROBIN	Finding	Commits	What users notice
A waiting request never waits for room inside a finish step	Found by the external review of JOB-5	`382588b`	With a full backlog, the scheduler no longer stops for good when a run ends
A sample finalised before still gets its later reads merged	JOB-6	`c93a711`	New batch BAMs and staged files reach `target.bam` on the next Live → Complete; nothing is skipped for the rest of the session
An SNP run first merges new reads, and reuses earlier calls only when they came from the same reads	JOB-6	`c93a711`	SNP calls follow the newest reads; a sample's next SNP run after the upgrade calls again, once; bulk SNP includes samples whose calls are older than their reads
A queued finalize is not also run by the GUI	JOB-7	`c93a711`	One merge per finalize request
An SNP run waits for reads still being added, fails when a read could not be added or a Clair3 region failed, and keeps staged files it could not add	JOB-6 (external reviews)	`e139331`, `d784753`	More SNP runs say "failed" instead of calling without some reads; a kept `_staging/_batch_<id>/` folder fails the sample's SNP runs until someone deals with it
A sample's CNV counts are read from disk by every run, under a per-sample lock, and written whole	CNV-2	`1f2c5b1`, `5e71fc6`	No batch is lost between workers; a sample's CNV runs take turns; unreadable counts, or an old-format counts file that can't be moved, fail the run instead of starting again from none
`master.csv` counts each BAM once, known by its contents, and an unreadable `master.csv` is kept and reported	JOB-2 (part 1)	`52b5169`	Rerunning a BAM, or a copy of it, no longer grows BAM passed/failed, bases and read counts; a new `bams_added` column; a `master.csv` from before has no list
A ledger of each BAM's steps: a BAM is taken in once (and only once complete), and after a restart only its unstarted steps run; the watcher's BAMs get the reference	JOB-2 (part 2), JOB-8, JOB-4 (partial files)	`4c60efb`, `80d457f`, `8b7cf02`	A restart no longer counts every BAM again; steps cut short by a stop, or that failed, show as failed files at every start; a new `robin_bam_ledger.jsonl`; a work folder from before has no ledger, so its BAMs run again once
A step's batch keeps its own list of BAMs, and Ray doesn't re-run a per-BAM step whose worker died	JOB-2's external review	`80d457f`	No BAM's data is added twice, or left out, through another step's batch or a retry; a step whose worker died now shows as failed
The Ray watcher sends waiting jobs on a timer, retries a failed send, sends before the Coordinator stops, and checks half-written BAMs again	JOB-4	`39039b7`, `8b7cf02`	The last BAMs of a run are analysed; unsent BAMs are logged by name (the ledger runs them at the next start); a BAM unchanged and incomplete for 10 minutes shows as a failed file
The target step shows each BAM whose coverage it could not add as a failed file	TGT-3 (noted by batch 8)	`3ab033c`, `c42c3d1`, `a932279`	A BAM left out of the coverage, or whose reads missed `target.bam`, shows in the failed files, with the folder that keeps its files; the step no longer says "success" for it. It shows again at every start, from the BAM ledger, whichever job found it
A coverage commit adds a batch to every file and hands over its batch BAM, or does none of it	TGT-3	`2582814`, `a932279`, `e7b9691`	A failure or crash part-way no longer leaves the coverage files at different generations: a list in `<sample>/_commit/` is finished by the next add-up or finalize. A new `_commit/` folder (gone after each commit) and `install.json`; a list that can't be read stops that sample's add-ups
The variant check names the regions it could not check, and fails the job	Noted by VAR-1's external review (batch 9)	`3ab033c`	Genes no longer vanish from the variant results on an error: the results list them, the job shows as failed, and the GUI names them. A BED line that isn't a BED row stops the check

Every change has a regression test in `tests/`. The details are in `CHANGELOG.md` under `[Unreleased]`. Add a row here each time a fix lands.

How this was done, and its limits

- **Reviewers.** Seven reviewers worked in parallel, one per area:
  - workflow and job handling
  - CLI, launcher and security
  - GUI screens
  - methylation and classifiers
  - CNV, fusion, target and SNP
  - PDF reports
  - packaging and design
- **Evidence.** Every finding was checked against the code, including whether anything actually calls it. Many were reproduced with small scripts on synthetic data, or by running the real ROBIN functions with Ray and NiceGUI stubbed out.
- **My own spot-checks.** I re-read about fifteen of the most serious findings myself; all of them held up. They are marked *(spot-checked)*.
- **Independent review.** GPT 6.1 Sol looked separately at:
  - sample identity
  - the lightweight variant check
  - target coverage and BAM accumulation
  - methylation read processing
  - the MGMT command
  - the CNV→master-BED handoff

  Each of its 11 findings has a test that runs the real ROBIN functions on small synthetic BAMs with pysam. I re-ran all 11 (11 of 11 reproduced) and read the code for five of them.
- **Not done:**
  - Nothing was run on real patient BAMs.
  - Ray, NiceGUI, torch and cnv_from_bam were not installed. pysam was used only for the independent review's tests.
  - The `nanoDX` and `hv_rapidCNS2` submodules were not checked out, so the R and NN code inside them was not reviewed.
- **Confidence labels:**
  - **Confirmed:** traced end to end, or reproduced.
  - **Plausible:** a strong reading of the code, but not fully traced (usually because it depends on timing or on Ray scheduling).

The reproduction scripts are in `2026-10-16-repro/`, with a table of which script shows which finding. The independent review's test suite is in `2026-10-16-repro/independent/`. They are scripts, not regression tests yet. A "Confirmed" label only becomes repeatable evidence once its script is turned into a test in `tests/`.

Read this first

The pipeline has a sound shape: a file watcher feeds a job graph, each sample keeps its state on disk, and the analyses are pluggable. The main problems are not crashes. **Most of the serious problems give wrong or**

stale results with no warning. Four patterns cause most of them:

1. **The methylation numbers fed to the classifiers are biased.** The merge averages percentages without weighting by coverage. matkit counts failed calls in the denominator. NanoDX throws away minus-strand calls.
2. **State lives in places that go stale.** There are caches inside long-lived Ray workers, the CNV control reference gets modified by every patient, the GUI keeps per-page state on a shared object, and there is one "waiting" slot per classifier for all samples.
3. **Errors are swallowed at every layer.** Failed jobs are counted as "completed", and error details are never sent to the GUI. This is why the "Error processing file" log line never showed up in the frontend.
4. **Inputs aren't tied to the right sample or batch.** A missing BAM falls back to another sample's BAM. Early BAMs never reach the SNP input. The master BED can drop the selected panel.

Only two tests exist (added after this review), and no CI runs them.

Fix first

#	ID	Problem	What goes wrong	Where	Size of fix
1	SEC-1 *(fixed in our fork)*	Ray dashboard open on all network interfaces, with no password	Anyone on the network can run code as the ROBIN user	`cli.py:1725-1731`, `2435-2449`	Small (two places)
2	SEC-8 *(fixed in our fork)*	MGMT runs the sample folder path through the shell	A crafted sample name runs commands; a space in a path breaks MGMT	`analysis/mgmt_analysis.py:836-838`	Small
3	VAR-9 *(fixed in our fork)*	A missing BAM for sample A falls back to sample B's BAM	Another patient's variant evidence is saved under this sample	`analysis/lightweight_gene_analysis.py:1562-1605`	Small
4	METH-1 *(fixed in our fork)*	Methylation % is averaged without weighting by coverage	Every classifier gets biased input, and adding depth doesn't make it better	`analysis/temp_utilities.py:285-391`	Small
5	CNV-1 *(fixed in our fork)*	CNV control reference is modified by every patient's reads	Real gains and losses fade over a run; false events appear in the next patient	`analysis/cnv_analysis.py:118-145`	Medium (build the control profile without patient reads)
6	JOB-1 *(fixed in our fork)*	One waiting slot per classifier type, shared by all samples	In multi-sample runs, a sample's final classification can silently never run	`workflow_ray.py:2462-2535`	Medium
7	GUI-1 *(fixed in our fork)*	Classification panel freezes if any classifier is disabled	GUI keeps showing the first result for the rest of the run	`gui/components/classification.py:1003-1021`	Small (two loops)
8	GUI-2 *(fixed in our fork)*	Refresh timers are switched off on any brief reconnect	An open page shows hours-old results, with no sign that they are stale	8 GUI components	Small
9	VAR-8 *(fixed in our fork)*, VAR-1 *(fixed in our fork)*, VAR-2 *(fixed in our fork)*, VAR-3 *(fixed in our fork)*	Variant summaries and ClinVar handling	Reference-only sites reported as pathogenic; IDH1 R132H-type alleles lost; "conflicting" shown as pathogenic; a stale index returns nothing	`lightweight_gene_analysis.py`, `snp_processing.py`, `clinvar_manager.py`, `reporting/sections/variants.py`	Small to medium

#	ID	Problem	What goes wrong	Where	Size of fix
10	TGT-1, TGT-2 *(fixed in our fork)*	Early BAMs never reach the SNP input; the master BED can drop the panel	SNP calls and IGV miss reads that the coverage numbers include	`analysis/target_analysis.py`, `cnv_analysis.py`, `master_bed_generator.py`	Medium
11	JOB-6 *(fixed in our fork)*	60-second idle timeout (a test value) left in	A sample is marked Complete and finalised too early; SNP calls cover only part of the data	`gui_launcher.py:420`	One line
12	METH-3, METH-4 *(fixed in our fork)*	matkit counts failed calls in coverage; NanoDX drops minus-strand calls	Methylation pushed down; about half the reads never reach NanoDX	`utilities/matkit.py`, `nanodx_analysis.py`	Medium
13	ERR-1	Failed jobs counted as "completed"; errors never sent to the GUI	Stale or empty results look current	`workflow_ray.py`, the handlers	Medium
14	JOB-2	Same BAM can be processed twice, for example after a restart	Coverage and counts doubled	`workflow_ray.py:4949-4977`	Medium
15	REP-1 *(fixed in our fork)*	PDF report uses the wrong Random Forest cut-offs	The PDF can say "High" where the GUI says "Medium"	`reporting/sections/classification.py:399`	One line

Before trusting more runs: compare ROBIN's merged parquet and classifier inputs with `modkit pileup --combine-strands` on one small real BAM. Make that comparison a regression test.

Upstream: SEC-1 (Ray dashboard), SEC-8 (MGMT shell command), SEC-9 (watched file names in command templates) and SEC-2 (MNP-Flex TLS) are security problems in upstream ROBIN. Report them to LooseLab privately, not in a public issue.

End-to-end data flow (for orientation)

1. `robin workflow` (`cli.py`) checks the model files, then starts `workflow_ray.run()` (the default) or `workflow_simple` (`--no-use-ray`), plus the NiceGUI app.
2. `RayFileWatcher` finds new BAMs. The `Coordinator` actor batches them per sample (`SampleJobBatcher`, `BATCH_CONFIG`) and dispatches Ray tasks. The job graph is `TRIGGERS`.
3. **preprocessing** (`bam_preprocessor`): reads the sample ID and read counts, and writes `master.csv`. It then fans out to bed_conversion, mgmt, cnv, target and fusion.
4. **bed_conversion**: `matkit.run_matkit` writes a per-BAM CpG parquet. `temp_utilities.merge_modkit_files` filters it to `parquet_filter.txt` and merges it into `<work>/<sample>/<sample>.parquet`.
5. **Classifiers**, triggered by bed_conversion:
   - Sturgeon (ONNX)
   - NanoDX and PanNanoDX (NN pickle)
   - Random forest (an R script in `hv_rapidCNS2`)

   Their input comes from `merge_bedmethyl.load_modkit_data`.
6. **Per-sample analyses:**

    - MGMT (methylartist)
    - CNV (`cnv_from_bam` against `HG01280_control_new.pkl`, with ruptures)
    - target coverage
    - fusion (`fusion_work`, clustering of supplementary reads)
7. target → `igv_bam` → `snp_analysis`, which the GUI triggers. It runs ClairS-TO in Docker (`:latest`), then SnpEff/SnpSift with ClinVar.
8. All state is CSV, parquet, pickle or npy files under `work_dir/<sample>/`. There is no database.
9. The GUI (`gui_launcher.GUILauncher`, `gui/components/*`) polls and unpickles those files. Optionally, MNP-Flex uploads to mnp-flex.org.
10. PDF reports (`reporting/report.create_pdf`) are built from the same sample folder.

---

1. Methylation numbers fed to the classifiers

METH-1 CRITICAL · **Confirmed (spot-checked)**: Cumulative `percent_modified` is a mean of means.

- **Where:** `analysis/temp_utilities.py:380-391`
- **What's wrong:** Each merge does `group_by(chrom, chromStart)`. It sums `valid_cov`, `n_mod` and `n_canonical` correctly, but sets `percent_modified = mean(...)` of the old aggregate and the new batch. So the newest batch always carries at least 50% of the weight. The summed counts sit in the same row but are never used.
- **Who reads it:** every classifier and MNP-Flex read only `percent_modified`. `merge_bedmethyl.collapse_minimal_bedmethyl` then takes the mean again, despite a comment there saying "weighted average".
- **Reproduced:**
    - 50 reads at 80%, then 1 read at 0%: stored 40.0%, true 78.4%.
    - Simulation of 500 CpGs at 80%, arriving at 20× depth one read per batch: NanoDX "methylated" calls (≥60%) fell to 81% of sites, against 99% with weighting.
- **Fix:** after summing, set `percent_modified = 100 * n_mod / valid_cov`.
- **Status:** fixed in our fork (`c50365a`): every write sums `n_mod` and `n_canonical` per CpG and sets `percent_modified = 100 * n_mod / valid_cov`. The case above now stores 78.43%. Checked against `modkit pileup` on synthetic reads (`dfe9f29`; result `5bcd583`). Still open in the original ROBIN.

METH-2 HIGH · **Confirmed**: The first write isn't aggregated, so random forest can see values over 100%.

- **Where:** `temp_utilities.py:285-333`
- **What's wrong:** When no cumulative parquet exists yet, the concatenated per-BAM rows are written as they are. A first batch can hold up to 20 BAMs, so there is one row per BAM per site.
- **What goes wrong downstream:**
    - The random forest path sums `score`, so the R script gets methylation of 200-300%.
    - Sturgeon counts the duplicate rows as separate sites.
- **Fix:** always run the same aggregation before writing (this is the same fix as METH-1).
- **Status:** fixed in our fork (`c50365a`): the first write goes through the same summing. Random forest's input is one row per CpG at the parquet's percent, never over 100 (`1c6c65f`). Still open in the original ROBIN.

METH-3 HIGH · **Confirmed**: matkit puts failed calls into `valid_cov`.

- **Where:** `utilities/matkit.py:1344, 1373-1385, 1132-1139`

- **What's wrong:** The total is incremented for every read before it is classified, so fails are counted. The result is `n_mod / (mod + canonical + fail)`; modkit leaves fails out of the denominator.
- **Example:** 6 modified, 2 canonical and 2 failed reads are stored as 60% instead of 75%. That moves the site across Sturgeon's 0.7 cut-off and onto NanoDX's 60% cut-off.
- **Plausible, secondary:** with combine_mods, the call uses max(P_m, P_h, P_C) rather than comparing P_m+P_h with P_C. This matters in brain tissue, which has high 5hmC.
- **Status:** fixed in our fork (`d304ded`): `valid_cov` is `n_mod + n_canonical` in the per-BAM parquet and in MGMT's BED. The "plausible, secondary" point is refuted: modkit 0.6.4's `--combine-mods` also takes the most likely class, so a read with 5mC 0.45 and 5hmC 0.45 is a failed call in both. Still open in the original ROBIN.

METH-4 HIGH · **Confirmed**: NanoDX and PanNanoDX drop every minus-strand CpG call.

- **Where:** `nanodx_analysis.py:342-349`, `utilities/merge_bedmethyl.py:490-501`
- **What's wrong:** matkit records the minus-strand call at C+1. The collapse step relabels the strand as "." but still groups on position, so the + and − rows of one CpG never merge. NanoDX's exact join on the 0-based C position then keeps only the + row.
- **Effect:**
  - About half the reads never reach NanoDX.
  - Sturgeon gets the two strand rows as equal votes, ignoring coverage: 3 reads at 100% count the same as 20 reads at 0%.
- **Fix:** shift − rows to position−1, then sum counts per CpG (the equivalent of `--combine-strands`).
- **Status:** fixed in our fork (`c50365a`): a minus-strand row moves onto its CpG's C and the two strands' counts are summed, as `--combine-strands` does. NanoDX/PanNanoDX, Sturgeon, random forest and MNP-Flex each get one row per CpG (`1c6c65f`). Still open in the original ROBIN.

METH-5 MEDIUM · **Confirmed**: Panel intersect plus inner join double-counts sites.

- **Where:** `temp_utilities.py:250-266`
- **What's wrong:** `parquet_filter.txt` has 976 pairs of overlapping intervals, including 90 exact duplicates. After the widening in METH-6 there are 8,238. PyRanges `intersect` returns one row per overlap, so the join duplicates the site.
- **Example:** in a CGCG, coverage is stored as 16 instead of 8.
- **Fix:** use a semi-join, and remove duplicates from the panel.
- **Status:** fixed in our fork (`dfd909f`, `c50365a`): exact duplicate panel rows are dropped, and rows are kept by a semi-join on the CpG's C, so each site counts once. Still open in the original ROBIN.

METH-6 MEDIUM · **Confirmed (spot-checked)**: `parquet_filter.txt` is already 0-based, so the v0.5 "1-based fix" is wrong.

- **Where:** `temp_utilities.py:159-179`, `docs/releases/v0.5.md:19`
- **Evidence:** cg13869341 is at `15864-15866` in this file, in the 0-based NanoDX BED and in the 0-based MNP-Flex BED. About 486k of its probes match the NanoDX file exactly.
- **What's wrong:** `Start - 1` widens every CpG to 3 bp, and the release note wrongly tells users their old parquets were wrong.
- **Also:** the first data row is `chr1 -1 0 cg22940546`, which has a negative start.
- **Fix:** read the file as plain 0-based BED, and correct the release note.
- **Status:** fixed in our fork (`dfd909f`): the file is read as listed (cg13869341, cg02984631 and cg14817997 read "CG" in hg38 at their starts), the 26 rows with a negative start are dropped, and `docs/releases/v0.5.md` carries a correction. 4,366 of the 900,614 rows are 1 bp long (3,091 "ch", 65 "rs" and 1,210 "cg" probes; of the 1 bp "cg" rows, 2 of 4 we checked sit one base off a CpG in hg38, and 12 others the final review checked sit

on one); every row's start is treated as a CpG's C. A "ch" row is a non-CpG C, so CpG calls give it no data (`b70d6a5`). Still open in the original ROBIN.

METH-7 MEDIUM · **Confirmed**: Classifiers never see coverage, and random forest gets a constant `cov=10`.

- **Where:** `merge_bedmethyl.py:72-189, 751-797`, `random_forest_analysis.py:283-346`
- **What's wrong:** The loader keeps only 5 columns. The random forest input invents `valid_cov=10`. So a site with 1 read counts the same as a site with 50.

METH-8 MEDIUM · **Confirmed**: Classifier failures are reported as success.

- **Where:** `sturgeon_analysis.py:585-588`, `merge_bedmethyl.py:93-102`, `nanodx_analysis.py:186-190, 366-397`, `mgmt_analysis.py:842-848`
- **Per classifier:**
  - **Sturgeon:** catches and logs the error, and appends no row.
  - **NanoDX:** appends an empty row with `number_probes=0`, and writes patient data to `errordf.csv` in the worker's current directory.
  - **MGMT:** an R failure only logs a warning, and the step returns True.
- **Effect:** the GUI keeps showing the previous result as if it were the latest.
- **Status:** fixed in our fork (`0241a47`). Sturgeon's background step no longer catches its error, so the Sturgeon job files it. NanoDX and PanNanoDX raise on a missing classifier or a prediction error: no row without scores, and no `errordf.csv`. MGMT's R step returns False on every R failure (a non-zero exit, an `Rscript` that can't start, a missing script), and the MGMT job files it, as it now does for an import error or a BAM sort failure; the MGMT BED file is still written. Each is a failed job, in the failed count, `failed_jobs.csv` and the GUI's errors (ERR-1). Not done: the classification panel still shows the last good result without marking it as old. Still open in the original ROBIN.

METH-9 LOW/MEDIUM · **Confirmed (mechanism)**: One BAM failing in a batch is skipped and its data deleted.

- **Where:** `temp_utilities.py:268-270`, `bed_conversion.py:252-262, 323-329`
- **What's wrong:** Per-file errors in the merge are logged and the loop goes on. All the per-BAM parquets are then deleted, including the failed one, and the job reports success.
- **Context:** this is the path behind the original "Error processing file ... ['Chromosome', 'Start']" message. That case is fixed on `main` (commit `d02add2`), and it lost no data. Commit `5be9e04` makes this log line include a traceback and safe diagnostics.
- **Status:** partly fixed in our fork (`34ceadd`): a file that fails is reported, with a `failed_jobs.csv` row of kind `file_failed` and a line in the GUI's errors, and the batch's other files still go on. Its data is still deleted with the other per-BAM files. Still open in the original ROBIN.

METH-10 LOW · **Confirmed**: The MGMT site table can show 0.43% as 43%.

- **Where:** `mgmt_analysis.py:336-347`; the GUI copy is at `gui/components/mgmt.py:226-235`
- **What's wrong:** It guesses "fraction or percent?" from whether any row is above 1. An unmethylated promoter at high depth fools the guess. The GUI copy has also drifted from the analysis copy.

METH-11 LOW · **Plausible**: The `bam_file_count` metadata undercounts.

- **Where:** `bed_conversion.py:300`, `temp_utilities.py:275`
- **What's wrong:** bed_conversion always passes `num_bam_files_seen=1`, and the count isn't updated when every file in a batch is skipped.

METH-12 LOW · **Plausible**: The panel filter cache is keyed by the panel file's basename only.

- **Where:** `temp_utilities.py:152`
- **What's wrong:** A different panel file with the same name reuses the old ranges. The cache pickle is also written non-atomically, outside the lock.
- **Status:** partly fixed in our fork (`dfd909f`): the cache's name carries a format version and the panel file's size and modification time, and it is written atomically, because every sample's old cache would otherwise have hidden METH-6's fix. Still open in the original ROBIN.

METH-13 HIGH · **Confirmed (reproduced; spot-checked)**: Implicit canonical calls in MM tags are ignored, which pushes methylation up. *(IR-05)*

- **Where:** `utilities/matkit.py:1287-1293, 1311-1333`
- **What's wrong:** matkit counts only the calls that `read.modified_bases` lists. In the MM tag's implicit mode (`.` or no marker), skipped C bases mean "unmodified", but matkit never counts them as canonical. A read with an empty implicit list (`C+m.;`) is skipped entirely.
- **Reproduced:** two reads cover `CGCG`, each with one confident modified call and one implicit canonical base. matkit stores 100% instead of 50% at each CpG.
- **Who it affects:** only BAMs that use implicit mode. In `?` mode, skipped bases are "unknown", and ignoring them is correct. To check which mode a BAM uses: `samtools view x.bam | head -1 | grep -o 'MM:Z:C+[a-z]*[.?]\?'`.
- **Fix:** follow the MM skip-mode rules when building each read's observations. Add test files for explicit, implicit and unknown modes, and compare with modkit.
- **Status:** fixed in our fork (`d304ded`, `b70d6a5`): in implicit mode, every CpG C the MM tag skips is an unmethylated call (CpG as the read reads it; the first fix counted every C, which the final review caught); a record with MM but no ML is skipped, as modkit does; `?` mode is unchanged; an implicit group with no positions now counts every C instead of skipping the read. Still open in the original ROBIN.

METH-14 MEDIUM · **Confirmed (reproduced)**: Whether a low-quality read is filtered depends on where its alignments lie. *(IR-06)*

- **Where:** `utilities/matkit.py:1263-1284`
- **What's wrong:** A read is added to the "exclude" set only when its low-QS primary record is reached. Supplementary records that come earlier in coordinate order have already been counted, and are never removed.
- **Reproduced:** the same QS=5 read keeps one methylation site when its supplementary alignment comes first, and none when its primary comes first. Both BAMs are coordinate-sorted.
- **Fix:** decide whether a read passes the QS filter before counting any of its alignments.
- **Status:** fixed in our fork (`d304ded`): only primary alignments count, as in modkit (its `--allow-non-primary` is off by default), so a low-QS read is dropped whatever the order of its records. Supplementary alignments no longer count toward methylation. Still open in the original ROBIN.

Not a problem:

- The 5hmC+5mC merge is by design.
- MGMT's 1-based to 0-based site handling is correct.
- Parquet writes are atomic.
- The cumulative merge is bounded by the panel size, not by total reads.

---

2. CNV

> **CNV-1** CRITICAL · **Confirmed (spot-checked)**: The cached control reference is modified in place and absorbs every patient's reads.

- **Where:** `analysis/cnv_analysis.py:118-145, 1181-1217, 1491, 1535-1597`
- **What's wrong:** `get_cached_ref_cnv_dict` returns one module-global dict, and it is passed straight to `cnv_from_bam.iterate_bam_file(copy_numbers=...)`, which adds reads into it (checked against the upstream Rust source). So in a long-lived Ray worker, the "control" grows with every BAM of every sample.
- **Simulated effect:**
  - Within one patient, a 3-copy gain reads +1.04 after the first batch, +0.49 when the patient has as many reads as the control, and +0.28 at 2.5× the control. A partial-purity gain drops below the ±0.4 call threshold, so events like 1p/19q or +7/−10 can disappear as the run goes on.
  - Across patients, a normal patient processed after another one showed mirror-image false events.
- **Fix:** build the reference profile from the stored control counts only, by re-binning them to the sample's bin width in numpy. Never pass a dict that must stay clean to `iterate_bam_file`.
- **Not enough on its own:** a `copy.deepcopy` per call is only a stopgap. It stops patients polluting each other, but each call still adds the current BAM to the copy, so the reference still contains that BAM's reads.
- **Status:** fixed in our fork (`b682da0`, `6ab6928`, `a06c4a2`, `2681f8b`). The new `analysis/cnv_reference.py` re-bins the stored control counts to the sample's bin width with the library's own formula, in 64-bit; the direct runner and `cnv_subprocess.py` use it in place of their second `iterate_bam_file` pass, and the cached control is a read-only mapping that the library refuses as `copy_numbers`. While fixing it we found that `cnv_from_bam` 0.4.4 sums bin chunks as 16-bit numbers with no overflow check: on the public control (12.7 M reads), a sample with up to about 24,000 reads gets bins whose control sums pass 65,535; at 4,000 and 20,000 reads the control's profile came out off by up to 4 and 17 copies. The 64-bit re-binning ends that too. One deliberate difference from the library: its median chunks the genome end to end in hash-map order, so a chunk can straddle two contigs; ours stays within each contig. Measured on the public control where nothing wraps, the two differ by up to 0.035 copies (at 25,000 sample reads; 0.015 at 50,000, 0.024 at 200,000). The sample's own profile still uses the library's median, so `CNV3.npy` can carry an offset of that size, an order of magnitude under the ±0.4 call threshold. A contig that only the sample has is no longer in `CNV3.npy`. Still open in the original ROBIN.

> **CNV-2** HIGH · **Plausible**: Per-sample CNV counts are cached per Ray worker and never invalidated.

- **Where:** `cnv_analysis.py:1137-1178, 1446-1484, 1245, 1620`, `workflow_ray.py:392-394, 2037-2068`
- **What's wrong:**
  - The cache only checks that the path matches.
  - CNV isn't serialised per sample.
  - The pickle is written non-atomically, and a load error silently resets the counts to `{}`.
  - The Coordinator's cache cleanup runs in the wrong process, so it does nothing.
- **Example:** batch 1 runs on worker A, batch 2 on worker B, and batch 3 on worker A again. Worker A uses its stale cache, so batch 2's reads are lost for good.
- **Fix:** always reload from disk under a per-sample lock, and write with tmp + `os.replace`.
- **Status:** fixed in our fork (`1f2c5b1`, `5e71fc6`, batch 10c). The per-worker cache is gone, and so is the Coordinator's cleanup of it. Each run reads the counts and the counter from disk, under a per-sample lock (`<sample>/_locks/cnv.lock`) held until the counts are saved, so a second run for the sample waits (up to 3 h, then it fails). The counts are written through a temporary file and a rename. Counts that can't be read fail the run, and the file is kept; they used to be reset to none. So does an old-format `update_cnv_dict.pkl` that can't be read or moved to the per-sample file: the original code skipped it with a warning, and the run then saved over its counts (found by the external review). Measured with stand-in Ray workers in separate processes: CNV-2's example (batch 1 on worker A, 2 on B, 3 on A) lost batch 2 before and keeps it now, and

two runs at once kept one batch before and keep both now. Not done: `cnv_analysis_counter.txt` is still written in place; in subprocess mode (off by default) a batch still keeps only its last BAM (CNV-3). Still open in the original ROBIN.

> **CNV-3** MEDIUM · **Confirmed**: Multi-BAM CNV adds the batch's last BAM twice.

- **Where:** `cnv_analysis.py:1515-1597, 1620`
- **What's wrong:** The loop has already accumulated every BAM, and the "final aggregated" call then adds the last one again. The result is saved, so the double count is permanent.
- **Status:** fixed in our fork (`391e159`): the loop's last result is the final one. Not fixed, and true before too: in subprocess mode (off by default, `USE_CNV_SUBPROCESS = False`), each loop call reloads the sample's counts from disk, which the loop never rewrites, so a batch keeps only its last BAM. Still open in the original ROBIN.

> **CNV-4** MEDIUM · **Confirmed**: The sex labels don't match the thresholds.

- **Where:** `cnv_analysis.py:677-689`, `classification_config.py:164`, `cnv_classification.py:195`, `gui/components/cnv.py:751-760`, `reporting/sections/cnv.py:365-373`
- **What's wrong:**
  - The estimator emits "Male (query X/Y ...)", which the threshold code treats as female.
  - The report compares against "XX"/"XY", which are never produced.
- **Effect:** for such males, a chrX gain needs >0.75 instead of >0.3, and chrY isn't skipped.
- **Fix:** use one shared XX/XY/UNKNOWN enum.

> **CNV-5** LOW · **Confirmed (by reading)**: Every chromosome gets a fake breakpoint at its end.

- **Where:** `cnv_analysis.py:574-580, 714-730`
- **What's wrong:** ruptures always returns the series length as its last change point. That adds about 24 spurious regions to the master BED.
- **Fix:** use `result[:-1]`.
- **Status:** fixed in our fork (`53165ed`). `run_ruptures` drops the terminator where it is produced, so those regions leave the master BED and `breakpoints_count` falls by about 24 per run. A chromosome whose only change point is the series end now yields no region at all, and when every chromosome does, `cnv_data_array.npy` and `breakpoints_NNN.bed` were not written, so the previous run's breakpoints stayed in the GUI and the master BED. Since `4032f8b` both are written empty instead; a detection that *failed* writes neither and logs a warning, so the last good breakpoints stay (`2ca41c7`). Still open in the original ROBIN.

The CNV summary card and the report's CNV problems are covered in sections 6 and 7.

---

3. Variants, fusions, target coverage and SNP input

Variants and ClinVar

> **VAR-1** HIGH · **Confirmed**: ClinVar records at the same position overwrite each other.

- **Where:** `analysis/lightweight_gene_analysis.py:228-229, 560-612`
- **What's wrong:** The lookup is keyed by `(chrom, pos)`, so only the last ALT allele at a position is kept.
- **Reproduced:** with IDH1 R132H (C>T) followed by C>A in the file, a patient who carries R132H shows "no_support". BRAF V600 and KRAS G12-type sites have the same problem.
- **Fix:** key by `(chrom, pos, ref, alt)`.

- **Status:** fixed in our fork (`502fd72`), as suggested. In ClinVar's GRCh38 file (2026-10-25), 67,276 of the records the variant check called pathogenic shared a position and were never checked. At IDH1 c.394 it kept R132S, so an R132C sample read "insufficient evidence". No `(chrom, pos, ref, alt)` key repeats in that file. The external review of the fix found that 13 pathogenic records have no alternate allele (ALT `.`). The evidence code failed on them and dropped the whole gene's results: PTEN and BRCA2 before the fix, and BRCA1 too once the position key stopped hiding its `.` record. They are now skipped and counted (`b97a709`). Still open in the original ROBIN.

VAR-2 HIGH · **Confirmed**: "Conflicting classifications of pathogenicity" counts as pathogenic.

- **Where:** `snp_processing.py:14-23, 132-136`, `lightweight_gene_analysis.py:260-265`, `reporting/sections/variants.py:61-75`
- **What's wrong:**
  - Substring matching on "pathogenic" also matches "Conflicting_..._of_pathogenicity". In the report, the CLNSIGCONF vote branch is therefore never reached, and it would crash if it were.
  - Also: `parse_vcf` groups with `dropna=True`, so records without ANN vanish from the table. In the test, 2 of 3 variants were shown.
- **Fix:** match exact CLNSIG tokens, use one shared helper, and `groupby(..., dropna=False)`.
- **Status:** fixed in our fork (`502fd72`), but not by CLNSIG alone.
  - **Why not CLNSIG alone:** BRAF V600E (7:140753336 A>T) and TERT C228T (5:1295113 G>A) have the germline label "Conflicting_classifications_of_pathogenicity", and they had counted only by the substring accident.
  - **The rule:** one shared helper (`analysis/clinvar_significance.py`) counts a record when CLNSIG is Pathogenic or Likely_pathogenic, ONC is Oncogenic or Likely_oncogenic, or SCI is Tier I or II. Labels are matched whole, after splitting on `|`, `/` and `,`. "Conflicting" alone never counts, and there is no vote (the user's choice).
  - **Measured on the 2026-10-25 file:** all three checks flag 367,875 records (the variant check flagged 549,186 before). 182,012 conflicting records drop out, and 701 records with only oncogenic or Tier I/II labels come in.
  - **Also done:** `parse_vcf` groups with `dropna=False`, the report's vote branch is gone, and the GUI's SNP table shows ONC and SCI.
  - **Not changed:** saved SNP tables and variant-check results keep their old flags until those steps run again.

  Still open in the original ROBIN.

VAR-3 HIGH · **Confirmed (reproduced with bcftools)**: A ClinVar refresh keeps the old tabix index.

- **Where:** `utils/clinvar_manager.py:130-131, 221, 300`
- **What's wrong:** After a new `clinvar.vcf.gz` is swapped in, `_ensure_tabix_index` returns early because the old `.tbi` exists. Region queries then return nothing with exit code 0, so pathogenic annotations silently disappear.
- **Fix:** always rebuild the index after replacing the file.
- **Status:** fixed in our fork (`8a37a7a`, `849e078`). `_ensure_tabix_index` takes a keyword-only `force`, and the two callers that have just written the `.gz` — `ensure_clinvar_files` and `update_clinvar_if_newer` — pass it, so the index is rebuilt with the file. A caller that did not touch the `.gz` keeps `force=False`, so a good index is not rebuilt on every start-up. A forced rebuild also deletes the old index before it starts, so a rebuild that fails cannot leave a stale index for the next start-up to accept as good — which would have brought this finding back behind an error from an earlier run. Still open in the original ROBIN.

VAR-4 HIGH · **Confirmed**: The report says "No pathogenic variants" when ClinVar annotation never ran.

- **Where:** `reporting/sections/variants.py:229-246, 306-335`, `target_analysis.py:3891-3940`
- **What's wrong:**
  - If the ClinVar DB is missing or SnpSift fails, plain snpEff output is copied to `snpsift_output.vcf`, and the report reads that as "annotated, none found".
  - The file is also truncated before each re-annotation.
- **Fix:** write atomically, store an annotation-status marker, and have the report say "ClinVar annotation unavailable".
- **Status:** fixed in our fork (`1fe4925`). The ClinVar step writes `##ROBIN_ClinVar=annotated` or `##ROBIN_ClinVar=unavailable;reason=…` after `##fileformat=` in `snpsift_output.vcf` and `snpsift_indel_output.vcf` (database missing, SnpSift not found, a non-zero exit, no output). Each file is written under a temporary name and swapped in, so a reader sees the previous file whole while SnpSift runs. The report names each file that was not annotated, with the reason, instead of "No pathogenic variants were identified."; a file from before this change reads "not recorded", and when it lists pathogenic variants the report says they carry ClinVar annotations and keeps their count (`2052fe3`). The SNP card shows the status too. ROBIN writes its own line rather than reading SnpSift's header, because real SnpSift could not be run here; the tests use a stand-in. A sample recorded unavailable is not annotated again by itself once ClinVar is available: force its SNP analysis to regenerate. Still open in the original ROBIN.

VAR-5 MEDIUM · **Confirmed**: Failed ClairS-TO regions are skipped silently.

- **Where:** `target_analysis.py:3549-3704`
- **What's wrong:**
  - A container that fails (including an OOM kill, exit 137) only logs a warning, and the merge reports "completed successfully for all regions".
  - Failed containers are never removed.
  - Docker and snpEff have no timeouts.

VAR-6 LOW · **Confirmed**: VCF→CSV uses the first variant as the header row.

- **Where:** `target_analysis.py:4143-4165`, `gui/components/coverage.py:3913-3933`
- **Effect:** the SNP/INDEL counts are one low, and a VCF with a single variant shows 0.

VAR-7 HIGH · **Confirmed**: `robin add-panel` keeps only the first region of each gene.

- **Where:** `cli.py:957`
- **What's wrong:** `drop_duplicates(subset=['gene'])` drops every later exon or tile of a gene. In the test, a panel with 3 EGFR rows and 2 TP53 rows became 2 rows. Coverage, SNP calling and fusion detection then ignore the dropped regions.
- **Status:** fixed in our fork (`1c8e95f`). The dedupe key is the whole region (`chrom`, `start`, `end`, `gene`), so every distinct region survives and an exactly repeated row still collapses. The second problem the reproduction script showed, under no finding ID — `read_csv` is given six column names, so an 8-column BED shifts every field and the function fails on `int('EGFR')` — is fixed too (`5af0a47`). It was worse than an error: a BED12 panel passed, printed "added successfully", and was saved as columns 7–10. The BED is now read line by line like the validator, keeping the first 4 columns. Still open in the original ROBIN.

VAR-8 HIGH · **Confirmed (reproduced)**: Reference-only samples are summarised as having pathogenic variants. *(IR-01)*

- **Where:** `analysis/lightweight_gene_analysis.py:904-933, 1451-1488`, `gui/components/coverage.py:5290-5310`

- **What's wrong:** Every covered ClinVar site in a region is counted as "found", even when its own evidence says `alternate_support=0, evidence_level="no_support"`. `export_json` then sets `pathogenic_status="present_with_good_coverage"` once coverage reaches 10 reads. The GUI's gene table and the HTML report (line 1233) show that status.
- **Reproduced:** 20 reads with only the reference C at a pathogenic C>T site. The pileup correctly shows zero alternate reads, but the summary reports one gene with pathogenic variants.
- **Fix:** keep "known site checked" separate from "variant supported". Base positive statuses and counts only on alternate-read evidence, and keep explicit "insufficient evidence" states.
- **Status:** fixed in our fork (`aeb5339`): a site counts with 3 or more alternate reads; 1–2 reads, an indel or multi-base change, or reads showing neither allele give "insufficient evidence" (`e5527a3`), and so does an SNV site with no usable bases, e.g. every read deleted there (`b26c410`), and a ClinVar site no read covers (`65ab945`), which used to drop out before export, so a gene whose only site had no reads vanished from the results. The unused HTML report (`generate_report`, no callers) still uses the old counting. Still open in the original ROBIN.

> **VAR-9** CRITICAL · **Confirmed (reproduced; spot-checked)**: A missing BAM for sample A falls back to sample B's BAM. *(IR-02)*

- **Where:** `analysis/lightweight_gene_analysis.py:1562-1605, 1665-1677, 1710-1730`; called from `gui/components/coverage.py:5882-5932, 6012-6069`
- **What's wrong:** If `<work>/A/igv/igv_ready.bam` is missing, the handler walks the whole work directory (`os.walk`) and takes the first `igv_ready.bam` it finds, from any sample. It then saves the result under A. The BED fallback searches the same way. The GUI enables the action based on A's `target.bam`, so this is reachable from the GUI.
- **Reproduced:** A has a reference-only BAM but no IGV BAM; B's IGV BAM has 20 alternate reads. A's results file contains B's 20 alternate reads.
- **Fix:** resolve the sample folder once and look only inside it. A missing input must give an explicit "not available" result. Record which BAM was used in the result.
- **Status:** fixed in our fork (`b9ffef8`, `d0e1975`). A missing input raises an error naming the expected path, which the GUI shows; results record `input_bam` and `input_bed`; older results files are not shown. When both result locations hold a trusted file, the newer one is shown (`f3f29cd`), and the panel's status timer stops when a run ends (`6acc03d`). Still open in the original ROBIN.

> **VAR-10** MEDIUM · **Confirmed (reproduced)**: Several BED intervals with the same name are reduced to the first one. *(IR-09)*

- **Where:** `analysis/lightweight_gene_analysis.py:656-671, 676-698`
- **What's wrong:** Regions are grouped by name, and one pileup is run over the first interval's coordinates, so variants in a gene's later exons are dropped. This happens even when the BED on disk keeps every interval (unlike VAR-7).
- **Reproduced:** two `TEST` intervals, each with a supported pathogenic site; only the first site is reported.
- **Fix:** key regions by coordinates, analyse each interval, then group results by gene name.
- **Status:** fixed in our fork (`20ea771`). Overlapping intervals are merged before the pileup. Still open in the original ROBIN.

Fusions

> **FUS-1** MEDIUM · **Confirmed (simulated)**: Master-BED SV breakpoints almost never reach the support threshold.

- **Where:** `fusion_work.py:1670-1672, 1775-1855`

- **What's wrong:** Reads are keyed by the midpoint of the whole alignment. The supplementary span is taken as `pos + query_length`, which is 1-based and covers the whole read. Supplementary records also pair with themselves.
- **Simulated:** 5 to 50 reads through one junction never put 3 or more reads in the same key, so zero breakpoints were written.

> **FUS-2** MEDIUM · **Confirmed**: The breakpoint BED is keyed to the CNV counter, so earlier breakpoints are dropped.

- **Where:** `fusion_work.py:3072-3090, 3879-3913, 4265-4272`, `master_bed_generator.py:85-115, 1073`
- **What's wrong:** Fusion reads `master_bed_breakpoints_{cnv_counter}.bed`, and the counter moves on almost every batch. So each new file holds only the new breakpoints, and earlier SV regions fall out of the dynamic targets.

> **FUS-3** MEDIUM · **Confirmed (deletion); Plausible (double append)**: Fusion accumulation has no lock and deletes staging files that failed to load.

- **Where:** `fusion_work.py:2175-2437`
- **What's wrong:** That BAM's fusion reads are lost. With `analysis_workers > 1`, supporting reads can be counted twice.

Target coverage, read counts and SNP input

> **TGT-1** HIGH · **Confirmed (reproduced; spot-checked)**: BAMs that arrive before the coverage threshold never reach the SNP input. *(IR-11)*

- **Where:** `analysis/target_analysis.py:1839, 1855-1890, 1999-2010`; batch entry at `2183-2223`; final merge at `2631-2658`
- **What's wrong:** Filtered batch BAMs are only made once some panel target passes `callthreshold`. Until then, coverage is still added up and the staged source BAM references are deleted. When a later batch passes the threshold, only that batch's BAMs are filtered, and the earlier ones are never revisited.
- **Reproduced:** three BAMs with one read each, where the third crosses the threshold. Coverage includes all three reads, but the batch BAM holds only the third. This needs only one worker, with no restart or timeout.
- **Fix:** filter and keep each BAM's panel reads when it arrives. Apply the threshold at calling time.
- **Status:** fixed in our fork (`d6c87ff`, `24ebbc8`): every BAM's records in the panel (plus the master BED's regions) go to a batch BAM when it arrives, with `samtools view -L`; the call threshold still decides when SNP calling may run. The final merge keeps what `target.bam` already holds, and its header lists the batch BAMs merged in, so a crash can't merge one twice. A batch BAM joins the sample folder only once its coverage is committed, so a commit that fails leaves both without it; IGV's copy (`igv/igv_ready.bam`) is rebuilt when `target.bam` gains reads; and a stale index is rebuilt (`888b5f0`). Found while fixing it: a second finalize deleted every new batch BAM unmerged. `target.bam` now holds the records overlapping the regions, not every record of a read whose primary overlaps. Still open in the original ROBIN.

> **TGT-2** HIGH · **Confirmed (reproduced)**: The CNV-generated master BED can replace the panel with breakpoint-only targets. *(IR-10)*

- **Where:** `analysis/cnv_analysis.py:1028-1039, 1730-1743`, `master_bed_generator.py:950-974, 1102-1129, 1187-1207`, `target_analysis.py:1860-1864`
- **What's wrong:** `cnv_handler` receives the panel, but the CNV save path looks it up only from fusion metadata. With fusion disabled, or before its metadata exists, it passes `target_panel=None`. The master BED is then written with CNV breakpoints but without the panel, and target extraction prefers it over the

configured panel. A later call with the right panel reuses the incomplete file, because the file already exists for that counter.

- **Reproduced:** a `demo` panel on chr1 with breakpoints on chr2: the master BED holds only chr2, and a panel read filters to zero reads.
- **Fix:** pass the panel explicitly from the handler. Always include the configured panel in extraction targets, and include the panel's identity in the cache key.
- **Status:** fixed in our fork (`063398d`, `d6c87ff`): `cnv_handler` passes its panel down to the master BED, and target extraction always reads the panel plus the master BED's regions. The cache key is unchanged: with the panel always included, a master BED written without it no longer costs reads. Still open in the original ROBIN.

> **TGT-3** MEDIUM · **Confirmed (reproduced; spot-checked)**: "Global coverage" adds up breadth across BAMs instead of measuring depth. *(IR-03)*

- **Where:** `analysis/target_analysis.py:1146-1148, 1817-1819`, `gui/components/coverage.py:6654-6679`, `reporting/sections/coverage.py:70-85`
- **What's wrong:** `covbases` (positions covered at least once) is summed per BAM, then divided by the genome length. Breadth doesn't add up across files, so the number depends on how the reads are split into BAMs.
- **Reproduced:** 10 reads over one of two 100-base contigs, so the true depth is 5×. As one BAM it reports 0.5×; as two BAMs, 1.0×.
- **How much it matters:** small early in a run, when reads in one BAM rarely overlap. Larger where they do.
- **Fix:** add up aligned bases (or length-weighted `meandepth`) for depth, and track breadth separately.
- **Status:** fixed in our fork (`d6c87ff`, `d752ce6`): each BAM gives its aligned bases per contig and per target (`bedcov -j`: deletions don't count, as in `samtools coverage`), and covered bases come from a union of covered stretches kept per sample. The GUI and the report show mean depth, and a target's length is end − start. Equal to samtools on all reads together, however they are split (`ee2fdd8`). In a folder written before, covered bases are left blank (unknown). The single "target coverage" number (and the enrichment and quality label made from it) used to sum bases and lengths over panel rows, so where rows overlap the overlap counted twice. In the AML panel, 1,180 rows merge into 910 regions and 3.59% of the summed length was counted twice; rCNS2 has no overlaps. Since batch 8, each BAM also counts its aligned bases over the merged panel, the sample keeps them in `target_union_coverage.csv`, and the number is samtools over the merged panel. A folder written before batch 8, or a sample with a BAM staged before it, keeps the row sum. So does a sample whose merged total stops matching its row totals (a commit that failed part-way, a panel changed mid-run, an unreadable staged part), so a partial merged total is never shown (an external review's findings). Each target's own depth is exact. An unreadable staged coverage file no longer deletes its BAM's set (batch 10b keeps it in its claim folder). Since batch 10e (`3ab033c`, `c42c3d1`, `a932279`), each BAM whose coverage was not added shows as a failed file, and so does one whose reads missed `target.bam`, where the step used to say "success"; the BAM ledger shows it again at every start. Since batch 10e's second part (`2582814`, `a932279`, `e7b9691`), a commit adds a batch to all of the sample's coverage files, and hands its batch BAM to finalize, together or not at all: the files are prepared in `<sample>/_commit/` with a list of the steps that put them in place, and a list a stop leaves is finished by the next add-up or finalize. All of this is still open in the original ROBIN.

> **TGT-4** MEDIUM · **Confirmed (reproduced)**: Two target accumulations at the same time overwrite each other's coverage. *(IR-04)*

- **Where:** `analysis/target_analysis.py:1792-1814, 1906-1934, 1999-2010`
- **What's wrong:** The lock is released between reading the old totals and writing the new ones. Two workers can read the same totals, and the second write replaces the first. Both report success and delete their staged files.
- **Reproduced:** two one-read BAMs in parallel give a total of 100 bases instead of 200. A single target worker avoids this.

- **Fix:** read, update and write the totals inside one locked step, and track which batches have been committed.
- **Status:** fixed in our fork (`d6c87ff`): one locked step reads the totals, adds the batch and writes every file atomically, and a staged BAM is claimed only once all its files exist. Two processes at once now give samtools' numbers. A finalize merges under its own lock, so a long merge can't make a commit give up (`888b5f0`). Committed batches aren't tracked: a batch whose worker dies after claiming it is lost, not counted twice (recovering it is left for later). Still open in the original ROBIN.

> **TGT-5** MEDIUM · **Confirmed (reproduced)**: Supplementary alignments are counted as extra reads. *(IR-07)*

- **Where:** `analysis/bam_preprocessor.py:352-386, 809-838`, `master_csv_manager.py:172-211`
- **What's wrong:** The "primary alignments only" check excludes secondary records but not supplementary ones. So read counts, yield and length summaries are inflated for split reads. The same counts feed the 50,000-read gate, so a BAM can be skipped as "too large" too early (traced in code, not reproduced).
- **Reproduced:** one 4-base read with one supplementary alignment is counted as 2 mapped reads and 8 bases.
- **Fix:** count reads and yield on primary records only, and keep a separate counter for supplementary alignments.
- **Status:** fixed in our fork (`7773e2f`): mapped and unmapped reads, bases, the pass/fail splits and yield count primary records only, and equal `samtools view -c -F 0x904` and `samtools stats`; `supplementary_reads` still counts the rest. The "has MGMT reads" flag also looks at primary records only. Still open in the original ROBIN.

---

4. Scheduling and job handling

> **JOB-1** CRITICAL · **Confirmed (spot-checked)**: One global waiting slot per classifier type.

- **Where:** `workflow_ray.py:998-1000, 2462-2535, 2807-2871`
- **What's wrong:** Classification is limited to one running job per type, plus one waiting job per type for all samples. If the slot is taken, a new request is dropped with no else branch.
- **Reproduced:** with samples A and B, Sturgeon ran for A twice and never for B.
- **Fix:** keep a "dirty" flag per (sample, type), and run dirty samples round-robin.
- **Status:** fixed in our fork (`f77bc5d`), as suggested. Each classifier type still runs one job at a time across all samples. It now keeps one waiting job per sample (the dirty flag), in arrival order. New data for a sample that is already waiting replaces its job, which keeps its place in line. When a run ends, the sample that has waited longest goes next. A run reads the sample's whole merged file, so a merged request loses nothing: a test checks that each sample's last run starts after its last `bed_conversion` ended. Still open in the original ROBIN.

> **JOB-2** HIGH · **Confirmed**: The same BAM can be processed twice.

- **Where:** `workflow_ray.py:4949-4977, 4116, 4155-4160, 4510-4551`
- **What's wrong:** The start-up scan doesn't add files to the watcher's dedup set. `add_watch_path` starts watching before it scans. There is no per-file record across restarts (`SQLiteStateTracker` exists but has no callers). Methylation counts, CNV and `master.csv` totals all just add up.
- **Effect:** a restart with the same `--work-dir` counts everything twice.
- **Fix:** keep a persistent per-file ledger, and share one dedup set.
- **Status:** fixed in our fork (batch 10d: `52b5169`, `4c60efb`, `80d457f`).
  - **The ledger (the user's choice, 2026-11-02):** the work folder keeps `robin_bam_ledger.jsonl`. It has one line when a BAM is taken in, and one when each of its per-BAM steps starts and ends. The worker writes

the start just before the step runs and the end just after it.

- **After a restart:**
  - A step that ended is skipped.
  - A step that never started runs: the BAM goes back through its preprocessing, told which steps to start.
  - A step that was running when ROBIN stopped is not run again, because it may have added its data. It is reported as a failed file that "may be incomplete", at every start.
  - A step that failed (the job's outcome, or a file that failed inside its batch) is not run again either, and is reported as failed at every start. A fail-only BAM's expected failure is not.
  - The non-Ray runner has no failed-file list, so it only logs these.
- **Taking a BAM in:** the start-up scan, the watcher and the GUI's "add folder" share one intake per session, in both runners. Watching starts before the scan, so a BAM that lands during the scan is not missed. A BAM is taken in only once it is complete.
- **How a BAM is known:** by its contents (`robin.bam_identity.bam_key`: its size and its first and last 64 KiB), so a copy of it or a moved folder is the same BAM.
- **`master.csv`** counts each BAM once (part 1). Re-running a BAM's preprocessing therefore doesn't count it again. A `master.csv` that can't be read is kept and reported, instead of being started again from zero. The list of BAMs it counts is `master_csv_bams.txt` next to it; `master.csv` holds only the last key, and the next update repairs the list if a stop came between the two writes.
- **The external review (Codex) of parts 1–2** found five problems, all fixed (round 2):
  - A BAM's steps share one set of job details, and each batch wrote its BAM list into its first BAM's. A step still waiting to run could then work on another step's batch: one BAM added twice, another left out. Older than this fix. Each batch now has its own.
  - Ray re-ran a task whose worker died (up to 3 times), so a step that had added a BAM's data added it again. Older too. The six per-BAM steps now fail instead.
  - With the list in one CSV cell, `master.csv` could not be read by Python's `csv` past 3,640 BAMs, and the target finalize then used `rCNS2`.
  - A cut-short report could be lost to a second stop, and a step that failed disappeared after a restart. Both are now reported at every start.
  - Found while briefing the review: a torn ledger line made the ledger unreadable one restart later. A torn line is now cut off before the next one is written.
- **A second external review** (Codex, of round 2 and JOB-4's part 3) found three more problems, all fixed (round 3, `8b7cf02`):
  - With `master.csv` deleted, or written again without its counts, and its list still there, the listed BAMs counted as done and their totals were silently missing. The update now stops with a failed file.
  - A step's failed files were found by counting the list before the step. But the BED step replaces the list, and in the non-Ray runner sibling steps share it at once, so a failure could vanish or land on a step that worked. Each entry is now tagged with the step that wrote it.
  - A failed-file row (an incomplete BAM, or an earlier failure) the Coordinator didn't get was lost for the session. The watcher now keeps it until the Coordinator has it, and the Coordinator shows a row once.
- **Why not the stricter plan:** first the user chose "re-run the unfinished steps exactly once". That needs every counting step to keep the BAMs it has added together with its totals. Mapped, that was six steps and seven batches, two of them large (target coverage and fusion), so the user chose the ledger.
- **Measured** with the real scan, watcher and worker wrappers, and a stand-in Coordinator: a finished BAM, or a copy of it, is not taken in again; only unstarted steps run; a cut-short or failed step is reported at every start and not run.
- **Not done:**
  - Per-sample steps (classifiers, IGV BAM, SNP, finalize) are not in the ledger. A stop can leave a sample's last run of one undone until its next data.

        - A work folder from before this change has no ledger, so its BAMs run again once.
        - Two sessions on one work folder at once are not supported.
        - A ledger that can't be read stops the Ray runner's start. The non-Ray runner only logs it for each BAM and analyses none.
        - With `--no-process-existing`, earlier BAMs' unstarted steps run only if the watcher sees those files again.
    - Still open in the original ROBIN.

> **JOB-3** **HIGH** · **Confirmed**: A failed handler runs again without `work_dir`, `reference` or `target_panel`.

- **Where:** `workflow_ray.py:704-742`
- **What's wrong:** The `try` wraps the handler call itself, so any exception falls through to the legacy `py_handler(job)`. The handler then runs a second time, writes output next to the BAM (inside the MinKNOW folder), and the real error is replaced by "target_panel is required".
- **Fix:** bind the arguments first, then call the handler once.
- **Status:** fixed in our fork with step 7 (`7a14955`; step 7's final review found it again). The arguments a handler takes are chosen first and checked with `inspect.signature(...).bind`. The handler is then called once, and the bare `handler(job)` call is used only when those arguments can't bind. A handler that raises runs once, its own error is the one recorded, and it writes nothing outside `work_dir`. Every registered handler gets the arguments it got before: 1,144 handler, job and metadata combinations were compared with the old dispatcher, with no difference. Still open in the original ROBIN.

> **JOB-4** **HIGH** · **Confirmed (flush and ordering); Plausible (partial files)**: The watcher delays or loses files.

- **Where:** `workflow_ray.py:4117-4193, 5020-5030`
- **What's wrong:**
    - The pending buffer only flushes when a new file arrives, and there is no timer, so the last file(s) of a run may never be analysed.
    - The flush at shutdown happens after the coordinator has stopped.
    - Submit errors are swallowed after the file has been marked processed.
    - There is no check that a file is complete, so a BAM copied in with `cp` or `scp` can be read truncated.
- **Status:** fixed in our fork (batch 10d: `4c60efb`, `39039b7`).
    - **Partial files** (the fourth bullet, with JOB-2): a BAM is taken in only once it ends with the BGZF end-of-file block, which its writer adds when it closes it.
    - **The buffer** (`39039b7`, part 3): a timer in the Ray watcher sends waiting jobs every 0.5 s, so none waits for a next file event.
    - **Failed sends:** a batch that can't be sent goes back to the buffer and is tried again, and each BAM's first failure is logged by name.
    - **Shutdown:** ROBIN stops watching and sends what waits before it stops the Coordinator, on the Ctrl-C path and at the end. Anything still unsent is logged by name; the BAM ledger runs it at the next start.
    - **Half-written BAMs:** the timer checks them again, so one that completes with no new file event (one the start-up scan found half-written, say) is taken in. One unchanged and incomplete for 10 minutes shows once as a failed file; it is still analysed if it completes later. A failed-file row the Coordinator didn't get is kept and sent again (`8b7cf02`).
    - **Not done:** the non-Ray runner has no buffer, and its half-written BAMs are only logged. Without watching (scan only), a BAM half-written at the scan is analysed at the next start.
    - Still open in the original ROBIN.

> **JOB-5** **HIGH** · **Confirmed**: Repeat SNP requests are silently skipped while the UI says "queued".

- **Where:** `workflow_ray.py:1730-1737, 2991-2997, 2363-2367`

- **What's wrong:** The dedup counter is never decremented (`mark_running` has no callers), so after the first `snp_analysis` for a sample, every rerun is dropped.
- **Status:** fixed in our fork (`f77bc5d`). For each sample, the Coordinator keeps the `snp_analysis` request running now and one waiting request. (The same rule covers a classifier job sent for one sample, which nothing in ROBIN does: their runs come from the JOB-1 path.) A request sent while one runs becomes the waiting one, in place of an older waiting one, because the newest carries the newest settings (such as forced regeneration). When the run ends, failed or not, the waiting one starts. `mark_running` is gone. Its external review found two more problems, fixed in `382588b`. First, SNP requests went through the sample batcher, whose batch ran with its first request's settings, and a request still in the batcher missed the waiting slot (three runs); per-sample requests now skip the batcher. Second, the waiting request was started as new work, which waits while the backlog is full, inside the ended job's finish step, whose pool slot stays busy until it returns; work already accepted now never waits for room. Not done: a job lost before it ends keeps its place, so that sample's later requests of that type wait for good (before, they were dropped). One way to lose a job is a queued job saved to disk that can't be read back. Still open in the original ROBIN.

> **JOB-6** `HIGH` · **Constant confirmed (spot-checked); impact Plausible**: The 60-second idle timeout is a test value.

- **Where:** `gui_launcher.py:420, 927-934, 7406, 7424-7426`, `target_analysis.py:2604-2618`
- **What's wrong:**
  - The code still says `completion_timeout_seconds = 60  # Set to 3600 for production`.
  - A gap of 60 s or more between BAMs marks the sample Complete and finalises it. Later SNP runs then skip the merge and use the old `target.bam`.
  - After a restart, unmerged batch BAMs are deleted.
- **Status:** fixed in our fork. First bullet (`c4592a0`, `b23ef4f`): the default is 3600 s, and `ROBIN_COMPLETION_TIMEOUT_SECONDS` overrides it (unset, empty, non-integer or below 1 all mean 3600). Third bullet: fixed by `24ebbc8` (TGT-1, batch 6), which this Status line missed until batch 10b: finalize merges every new batch BAM into the existing `target.bam` and deletes none unmerged. Second bullet (`c93a711`, batch 10b): the GUI's in-memory set of finalised samples is gone, so every Live → Complete starts a finalize, and the on-disk "nothing to merge" check also counts staged files. The SNP job first merges new batch BAMs and staged files. All three SNP buttons reused existing calls whenever they existed; a finished call now records which `target.bam` it read (`clair3/target_bam_source.json`: its batch list, size and time), and earlier calls are reused only when that matches and nothing waits to be merged. Bulk SNP and the GUI's "SNP complete" poller use the same check. An external review of `c93a711` found four gaps, fixed in `e139331`: (1) staged files a worker had claimed (moved into `_staging/_batch_<id>/`) were invisible to the merge and the checks; finalize now waits for that worker, up to 5 minutes. (2) An SNP run went ahead after a merge that missed reads (a lock timeout, unreadable staged files, a batch BAM that could not be written), and those files were deleted. Now the run fails, and the files are kept in their folder with an `incomplete.txt`. (3) A failed Clair3 region was skipped, and VCFs from an earlier run were used when this run wrote none; each region must now finish and write both VCFs during the run. (4) The batch list alone did not change when `target.bam` was rewritten another way; the record now also holds its size and time (every way ROBIN writes `target.bam` changes them; a replacement from outside that keeps both would not be noticed). A second review of `e139331` found two smaller gaps, fixed in `d784753`: a VCF from a run that had just ended could pass the time check, so each run now writes Clair3's files into its own folder (`clair3/run_<id>/`); and a staged set missing a part was skipped quietly, so finalize now lists it and the SNP run fails. Not done: when the workflow's job types are known and their outputs exist, the background scan's first Complete check marks a timed-out sample Complete without a finalize (its batch BAMs then wait for the next SNP run or a manual finalize; which scan sees the change first in a real run is not measured); the coverage panel shows SNP calls without saying they are older than `target.bam`; a kept `_staging/_batch_<id>/` folder is not recovered automatically (its `incomplete.txt` says whether its coverage was counted), so the sample's SNP runs fail until someone deals with it; the finalize job itself still fails only on an `"error"` result. Still open in the original ROBIN.

JOB-7 HIGH · **Double run Confirmed; corruption Plausible**: `target.bam` finalisation runs twice.

- **Where:** `gui_launcher.py:7422-7512`
- **What's wrong:** `finalization_succeeded` stays True after the Ray job is queued, so the GUI also runs the finalisation inline. The two copies delete each other's temp and batch files.
- **Fix:** run the inline finalisation only when the submit fails, and add a per-sample lock.
- **Status:** fixed in our fork (`c93a711`, batch 10b). The GUI merges itself only when there is no runner or the job can't be queued. The per-sample lock came with TGT-1 (`24ebbc8`, batch 6): finalize merges and deletes under a `finalize` file lock, with unique temp names. Measured: two processes finalising one sample at once, with a pause before each merge so they overlap, gave 0 bad `target.bam`s in 40 trials with the lock, and 39 of 40 without it (one finalize failed on a batch BAM the other had deleted). So since batch 6 the double run was wasted work, not corruption. Still open in the original ROBIN.

JOB-8 MEDIUM · **Confirmed**: Files submitted by the watcher have no `reference` in their metadata.

- **Where:** `workflow_ray.py:4155-4185` vs `3756-3771`
- **What's wrong:** Files found at start-up get the reference and target panel; files that arrive later don't. So mgmt, target, CNV and fusion behave differently depending on when a BAM appeared.
- **Status:** fixed in our fork (`4c60efb`, batch 10d-2, with JOB-2). The scan and the watcher build a file's jobs with one helper: work folder, reference, target panel, fail-only, and the BAM's key. The watcher asks the Coordinator for the reference once. Still open in the original ROBIN.

JOB-9 MEDIUM · **Confirmed (opt-in setting)**: A job timeout releases its pool slot twice.

- **Where:** `workflow_ray.py:3575-3599` vs `3527-3534`
- **What's wrong:** With `ROBIN_JOB_TIMEOUT_SECONDS` set, each timeout adds a phantom slot, so a pool limited to 1 job runs jobs at the same time.

JOB-10 MEDIUM · **Plausible**: Shutdown kills jobs mid-write and drops waiting jobs.

- **Where:** `workflow_ray.py:3281-3363, 5020-5054`
- **What's wrong:** A Ctrl-C during a CNV save leaves a truncated pickle, and the next start silently resets the CNV history.

JOB-11 MEDIUM · **Confirmed**: The default Ray path ignores several CLI options.

- **Where:** `cli.py:2419-2537`
- **What's wrong:** `--no-gui`, `--commands`, `--deduplicate-jobs`, `--queue-priority` and `--show-priorities` have no effect. For example, `--show-priorities` starts a full run, and `--no-gui` still opens the GUI.

JOB-12 LOW · **Plausible**: Race conditions when `analysis_workers > 1`.

- **What's wrong:** MGMT's `mgmt.bam` accumulation and file numbering have no lock, and neither does the panel filter cache pickle. The default of 1 worker avoids this.

JOB-13 LOW · **Confirmed (latent)**: All job types for a file share one context object.

- **Where:** `workflow_ray.py:2554, 2947-2964`
- **What's wrong:** `_batched_job` is overwritten by each type's batch. This is harmless today only because every type's batch holds the same files.

---

5. Errors that never reach the user

This is why "Error processing file" showed up in the backend log but never in the GUI.

> **ERR-1** HIGH · **Confirmed**: Errors are lost at five layers.

- **Where:** `workflow_ray.py:3515-3521, 801-809, 4733-4928, 755`
- **Layer by layer:**
  1. The merge swallows per-file errors and reports success (METH-9).
  2. Handlers record errors under `"<type>_analysis"`, but the Pool only checks the `job_type` key, and the `finally` block adds `{"status": "success"}`. So **for 8 of 13 job types, a failed job counts as completed**: it isn't in `failed_jobs.csv` or in the failed count.
  3. `_publish_gui` sends only counts. Nothing ever sends `ERROR_UPDATE`, even though the GUI has handlers for it.
  4. `send_gui_update` calls from inside Ray workers find no GUI in that process and are dropped. This includes BAM alignment warnings such as "unaligned BAM" or "<1% mapped".
  5. The TypeProcessor path (preset `high`) has no error detection at all.
- **Fix:** treat any `add_error` during a job as a failure, return per-file failures from the merge, forward failures from the driver to the GUI, and route worker notifications through the Coordinator.
- **Status:** fixed in our fork. Layers 2 and 5: any error a job files while it runs fails that job, whatever name it is filed under, on both schedulers and in the non-Ray runner; an error an earlier job left in the same context doesn't, and a fail-only BAM submission's error stays expected (`9bfe410`, `c52d86c`). A failed job no longer starts the jobs after it (on `standard` only `target` has one, `igv_bam`; on `high` and with `--no-use-ray` a failed `bed_conversion` or `preprocessing` now stops its next jobs too). The non-Ray runner marks fail-only submissions by the Ray runner's two rules (`7a14955`). Layers 3 and 4: the Coordinator keeps the recent failures and the messages workers sent with their jobs, and the GUI process sends them on as `ERROR_UPDATE` and as their own update types; "Recent errors" shows the newest five (`ee7e3dd`). A worker's message arrives when its job returns; a crashed job's messages go with that worker's next job, or not at all. One bad event no longer blocks those behind it. Updates go out at a priority the GUI neither drops when busy nor merges. Events lost from the 200-event log between two polls are counted in one GUI line, and the error counts are refreshed anyway (`2052fe3`, `4ab371b`). Layer 1: the merge and the parallel BAM step return their failed files, each one a `failed_jobs.csv` row of kind `file_failed` and a line in the GUI's errors (`34ceadd`; METH-9). CNV's batch error is filed too. Each failure was injected on purpose through a real local Ray Coordinator on both schedulers (`55eee97`). Still open in the original ROBIN.

> **ERR-2** MEDIUM · **Confirmed**: Import failures are swallowed and reported with the wrong cause.

- **Where:** `analysis/__init__.py:15-97`, `cli.py:55-81`, `workflow_ray.py:124-200`, `nanodx_analysis.py:24-38`, `temp_utilities.py:30-34`
- **Example:** a Sturgeon API change makes every Sturgeon job fail with "No implementation available for job_type='sturgeon'", with no reason given.
- **Missing submodules:** an empty submodule folder isn't detected.
- **Fix:** import lazily per job type, and print every import error at start-up.

> **ERR-3** · **Confirmed**: The codebase has 1,325 broad `except Exception` or bare `except` blocks, and 404 of them do nothing but `pass` or `continue`.

---

6. GUI

> **GUI-1** CRITICAL · **Confirmed (spot-checked)**: The classification panel stops updating when any classifier is disabled.

- **Where:** `gui/components/classification.py:1003-1021`
- **What's wrong:** The loop goes over all four tools in `tool_to_file`, but `charts` only has entries for the enabled ones. So `charts['PanNanoDX']` raises KeyError on every tick after the first. The panel keeps showing the first result, while the summary card moves on.
- **Fix:** in both loops of `_refresh_classification` (lines 1003-1008 and 1019-1021), loop over `charts` instead of `tool_to_file`, or skip tools that aren't in `charts`. The third loop (1041-1047) is already guarded.
- **Status:** fixed in our fork (`50da100`). The mtime scan moved into a module-level `classification_files_to_refresh` keyed by the tools present in `charts`, and the update loop and the log line iterate `charts` too, so a run with a classifier disabled keeps refreshing. Still open in the original ROBIN.

> **GUI-2** CRITICAL · **Confirmed (against NiceGUI 3.6 source)**: A brief websocket drop permanently stops every refresh timer.

- **Where:** `summary.py:352`, `classification.py:1068`, `coverage.py:6917`, `mgmt.py:721`, `cnv.py:2467`, `fusion.py:3421`, `mnpflex.py:860`, `bed_coverage.py:456`
- **What's wrong:** Every component calls `timer.deactivate()` in `on_disconnect`, which in NiceGUI 3 also fires on reconnect, and nothing reactivates the timers.
- **Effect:** after a Wi-Fi blip, laptop sleep or proxy timeout, the page shows hours-old results.
- **Fix:** use `client.on_delete`.
- **Status:** fixed in our fork (`7ff93b1`). All nine call sites use `client.on_delete` — the eight above plus `gui_launcher.py:5130`, which has the identical defect and is not listed here. The six theme-sync unregisters in those callbacks were the same bug class and are fixed with them. The trade: a page that really was closed is torn down up to 60 s later, after ROBIN's `reconnect_timeout`. No test proves a timer survives a real reconnect; `tests/test_gui_timer_cleanup.py` is lint-shaped. Still open in the original ROBIN.

> **GUI-3** HIGH · **Confirmed (MGMT simulated)**: A second viewer of the same sample misses updates.

- **Where:** `mgmt.py:388-405`, `cnv.py:1989-2010`, `coverage.py:6497-6531`, `gui_launcher.py:491-499`
- **What's wrong:** The "last seen mtime" markers live on the one shared `GUILauncher`. The first browser to see a new file records it, and the others then think nothing changed.
- **Reproduced:** after a new batch, browser A showed "Methylated" and browser B still showed "Unmethylated".

> **GUI-4** HIGH · **Confirmed (spot-checked)**: The CNV summary card shows the batch counter as "Gained".

- **Where:** `summary.py:1245-1292`
- **What's wrong:** It reads `cnv_analysis_counter.txt` as the number of gains. After 37 batches the card says "Gained: 37, Lost: 0" for every sample. The other files it looks for are never written.
- **Status:** fixed in our fork (`e677991`). The counter file has left the candidate list, and `gained` / `lost` default to `None`, which the card renders as "Not available" — the idiom it already uses for genetic sex, bin width and variance. Because nothing in `src/` writes a per-sample gain/loss count, that is what every sample now shows; a real `cnv_results.csv` would still supply real counts. Still open in the original ROBIN.

> **GUI-5** HIGH · **Confirmed (by reading)**: Per-page state is shared by all browsers.

- **Where:** `gui_launcher.py:2396, 2732, 3155-3163, 7289-7317`

- **What's wrong:** The sample table, selection and filters are stored on the singleton.
- **Example:** user B opening `/live_data` clears user A's selection, so A's Export gives B's samples.

GUI-6 MEDIUM · **Confirmed (path); race Plausible**: The GUI rewrites analysis output files.

- **Where:** `summary.py:1612-1626` → `fusion.py:307-404`, `mgmt.py:594-603`
- **What's wrong:** Every 30 s, for each browser, it rewrites `fusion_summary.csv`, `fusion_results.csv` and `sv_count.txt` non-atomically, using its own counting method. MGMT also overwrites `{N}_mgmt.pkl`.
- **Fix:** make the GUI read-only over sample folders.

GUI-7 MEDIUM · **Confirmed**: The summary card and the coverage panel use different coverage thresholds.

- **Where:** `summary.py:1180-1187` vs `coverage.py:6681-6699`
- **Example:** at 12× the card says "Moderate" while the panel says "Low".

GUI-8 MEDIUM · **Confirmed**: The GUI fusion check uses a different minimum support.

- **What's wrong:** It re-validates with min_support=4, while the analysis and `fusion_summary.csv` use 3. So the summary and the fusion tab can disagree.

GUI-9 LOW · **Confirmed**: The MGMT card's CpG count says "Not available" or is wrong.

- **Where:** `summary.py:1338-1388`
- **What's wrong:** The helper is defined in one branch only, so the other branch hits a NameError.

GUI-10 LOW · **Confirmed**: The "global" theme and metrics timers die with the first client.

- **Where:** `theme.py:127-134, 437-443`
- **What's wrong:** They are bound to the first client. Also, other users' charts follow the first user's dark-mode setting.

---

7. PDF report

REP-1 HIGH · **Confirmed (spot-checked)**: Random Forest confidence uses the default cut-offs.

- **Where:** `reporting/sections/classification.py:399`
- **What's wrong:** It looks up `"random forest"`, but the key is `"random_forest"`, so the default 80/50 is used instead of 85/65. At 82%, the PDF says "High" and the GUI says "Medium".
- **Fix:** map display names to config keys, and raise an error on an unknown classifier.
- **Status:** fixed in our fork (`229247f`, `f50ad61`). `classification_config` gained `CLASSIFIER_DISPLAY_NAMES` and `config_key_for_classifier()`, which resolves a display name to its key and raises on an unknown one; the report's one call site goes through it, so 82% bands as "Medium" in both the PDF and the GUI. `get_confidence_status` itself stays permissive, because `summary.py` passes section anchors ("coverage", "cnv", "mgmt", "fusion") that need the default fallback. Still open in the original ROBIN.

REP-2 HIGH · **Confirmed**: CNV gene counts and plot selection use an old ±1 SD rule.

- **Where:** `reporting/sections/cnv.py:122-160, 580-620, 988-1005`
- **What's wrong:** The GUI switched to fixed thresholds; the report still uses the old rule.
- **Reproduced:** on a CNV-neutral genome, the report listed 20 "gained" and 17 "lost" genes and plotted all 23 chromosomes, while its own events table was empty.

REP-3 HIGH · **Confirmed**: The fusion table shows the weakest breakpoint cluster.

- **Where:** `reporting/sections/fusion.py:249-262`
- **What's wrong:** The last row wins, and the rows are sorted strongest first. For example, KIAA1549-BRAF shows a 4-read cluster in the PDF, while the CSV export shows 25 reads.

REP-4 HIGH · **Confirmed**: Gene pairs are split on "-", which breaks hyphenated genes.

- **Where:** `reporting/sections/fusion.py:53, 121-127, 254, 561`
- **What's wrong:** Names like `EGFR-AS1` and `H3-3A` get cut in two. For example, EGFR-SEPTIN14 becomes the network "EGFR,EGFR - AS1", and SEPTIN14 disappears.
- **Fix:** carry gene1 and gene2 as separate fields.

REP-5 HIGH · **Plausible**: Report plotting shares global matplotlib state with GUI threads.

- **Where:** `reporting/plotting.py:340-430`, `analysis/methylation_wrapper.py:13-25, 256-293`
- **What's wrong:** The report uses pyplot's global state in a NiceGUI worker thread, while the GUI's MGMT refresh monkeypatches `Figure.savefig` process-wide.
- **Effect:** a plot can be swallowed, or another figure (possibly another sample's) embedded.
- **Fix:** use the object-oriented `Figure` API, or run report generation in a subprocess.

REP-6 MEDIUM · **Confirmed**: Missing fusion data is reported as "No fusion networks identified".

- **Where:** `reporting/sections/fusion.py:133-197, 425-441, 640-645`
- **What's wrong:** "Not run", "unreadable" and "none found" all print the same negative result. The methods text also states different thresholds from the code (>50 vs >49, >200 bp vs >249 bp).
- **Status:** fixed in our fork (`c60f817`, `6aad4ba`, `11573aa`, then `2052fe3`, `7a14955`, `97b34ee` and `4ab371b` after its final reviews). The report says "Fusion analysis has not been run for this sample.", "The fusion results could not be read." or "No fusion candidates were identified.", the last only when the step ran and nothing passed its filters. The target panel and the genome-wide results are judged apart, so one type's missing results are said even when the other has candidates. A build that fails leaves a `.failed` record beside its pickle. The type then reads "could not be read", or its earlier candidates show with "The latest ... fusion build failed; these results are from an earlier run.". The next build rebuilds that pickle from the candidate CSV even with no new reads. The GUI's fusion panel uses the same per-type state. A fusion job that fails before it builds records the failure for both types. A candidate file that can't be read fails the job and names the file (batch 7b: `8c795c6` onward; found by an external review). Before, it was logged and skipped: its reads were lost, and the type could read "none found". Staged files and accumulated parts are written under a temporary name and renamed into place, so a reader never meets half a file. Not done: two accumulations of one sample at once (no lock; step 8, JOB-1/JOB-2). When nothing passes, the step writes empty processed pickles, so an earlier run's candidates can't show as the latest; a folder from before this change is read from its `sv_count.txt` and candidate CSVs. A failed build fails the fusion job, still writes the step's other outputs, and is rebuilt on a later run. The methods text is built from the code's settings: mapping quality of at least 50, a span of at least 250 bp, at least 3 reads per gene pair, and 4 reads at the same breakpoint for a fusion to be reported. Still open in the original ROBIN.

REP-7 MEDIUM · **Confirmed**: CNV plot y-limits hide focal amplifications.

- **Where:** `reporting/plotting.py:264-267, 297, 356-357, 420`
- **What's wrong:** The limit is mean + 2 SD. An EGFR amplification at about 30 copies falls off the axis (the limit is about 8.6).

REP-8 MEDIUM · **Confirmed**: At low CNV resolution, the section crashes after half the summary is built.

- **Where:** `reporting/sections/cnv.py:689-723, 805`
- **What's wrong:** `events` is unbound, so the summary shows the CNV heading with no events and no "resolution insufficient" note.

REP-9 MEDIUM · **Confirmed**: A coverage file being written can abort the whole report.

- **Where:** `reporting/sections/coverage.py:28-31, 55-113`, `report.py:59-67, 142-144`
- **What's wrong:** The coverage CSVs are read in the constructor, outside the per-section error handling.

REP-10 MEDIUM · **Plausible**: The report reads live, non-atomic files with no snapshot.

- **Where:** `reporting/sections/cnv.py:418-433, 492-496`, `reporting/sections/classification.py:339-413`
- **What's wrong:** CNV.npy, CNV_dict.npy and CNV2.npy can come from different iterations. A stale `PanNanoDX_scores.csv` from an earlier configuration can appear in the PDF but not in the GUI.

---

8. Security and privacy

SEC-1 CRITICAL · **Confirmed (spot-checked)**: The Ray dashboard and Jobs API listen on 0.0.0.0 by default, with no authentication.

- **Where:** `cli.py:2435-2449, 1725-1731, 2302-2306`, `workflow_ray.py:5161`
- **What's wrong:** Anyone on the network can POST a job to `http://<host>:8265/api/jobs/` and run shell commands as the ROBIN user, with access to all BAMs and reports. This is the ShadowRay / CVE-2023-48022 class of issue.
- **Fix:** default to `127.0.0.1` and make the dashboard opt-in.
- **Status:** fixed in our fork (`3623e93`). Still open in the original ROBIN.

SEC-2 HIGH · **Confirmed**: MNP-Flex TLS verification is hard-coded off.

- **Where:** `gui_launcher.py:7180, 7225-7231`, `gui/components/mnpflex.py:92-147, 829-855`, `utils/mnpflex_client_standalone.py:34`
- **What's wrong:**
  - The credentials (the `client_secret` defaults to `"SECRET"`) and the patient methylation BED are sent without certificate checks, and the downloaded classification is shown as a result.
  - The upload starts automatically, once per open tab.
  - On failure, patient data is left on the third-party server.

SEC-3 HIGH · **Confirmed (reproduced)**: Partial model downloads are kept and accepted.

- **Where:** `utils/model_updater.py:121-152, 290-306`, `utils/model_checker.py:181-187`
- **What's wrong:** Downloads stream straight to the final filename, and the start-up check only tests `size > 0`. The models are pickles, so a swapped file means code execution.
- **Fix:** download to `.part`, verify the SHA-256, rename, and re-verify at start-up.

SEC-4 MEDIUM · **Confirmed**: The GUI listens on 0.0.0.0 over plain HTTP.

- **Where:** `cli.py:2284-2289`, `gui_launcher.py:535-546, 1845-1863, 2122-2131`
- **What's wrong:**

- `storage_secret="robin"` is a constant.
- Logins aren't throttled, and the argon2 check runs on the event loop.
- There is an open redirect via `redirect_to`.
- The "local client" check trusts the Host header.

SEC-5 MEDIUM · **Mechanism Confirmed; exploit Plausible**: The auth middleware lets some routes through without checks.

- **Where:** `gui_launcher.py:549-553, 2113-2117`
- **What's wrong:**
  - `/_nicegui*` and `/api/v1/*` are exempt, and `ui.download` URLs are predictable.
  - `/api/download/{sample_id}/{filename}` accepts `..` (one level up).

SEC-6 MEDIUM · **Confirmed**: Patient identifiers can be decrypted from the date of birth alone.

- **Where:** `gui_launcher.py:98-131, 8788-8806, 4407-4431`, `reporting/report.py:193-204`
- **What's wrong:**
  - The key comes from PBKDF2(DOB, fixed salt). That is about 36,500 possible keys, which takes about 20 s on a 72-core server.
  - The sample ID is an unsalted MD5, and it is sent to MNP-Flex.
  - PDFs with plaintext identifiers are left in the sample folder.

SEC-7 MEDIUM · **Plausible**: About 30 sites load pickles from results folders.

- **Examples:** `gui/components/summary.py:1221`, `reporting/sections/cnv.py:433`, `fusion_work.py:2601`
- **What's wrong:** Opening a results folder, or building a PDF from one, received from someone else can run code.

SEC-8 HIGH · **Confirmed (reproduced; spot-checked)**: MGMT runs the sample folder path through the shell. *(IR-08)*

- **Where:** `analysis/mgmt_analysis.py:836-838`; called at lines 1081 and 1393. Sample IDs come from BAM read-group metadata (`bam_preprocessor.py:260-275`).
- **What's wrong:** The Rscript command is built as one f-string with `sample_dir` and other paths unquoted, and run with `subprocess.run(cmd, shell=True)`. A sample ID becomes part of that path with no shell-safe handling.
- **Reproduced:** a sample folder named with a harmless `$(touch …)` marker. Running the real MGMT command path creates the marker file, so the shell ran it. A space in any path also splits the arguments and breaks MGMT.
- **Fix:** pass an argument list with `shell=False`. Separately, validate sample IDs before using them in paths.
- **Status:** the shell call is fixed in our fork (`2d3577e`, `0873744`). Sample IDs are not validated yet. Still open in the original ROBIN.

SEC-9 MEDIUM · **Confirmed (tested)**: Watched file names are pasted into the operator's shell command. *(Found 2026-10-30, while planning the fix for SEC-8.)*

- **Where:** `watcher.py:116` (`FileChangeHandler._run_command`) and `workflow_simple.py:1756` (`command_handler`, used by `--commands`).
- **What's wrong:** Both do `template.replace("{file}", path)` and run the result with `shell=True`. The template is the operator's own command, so a shell is intended, but the file name comes from the watched folder. A file named `a;rm -rf ~.bam` runs `rm`.

- **Tested:** with the old code, file names containing `$(touch …)` created the marker file, in bare, `"{file}"` and `'… {file} …'` templates.
- **Fix:** keep the shell, but never paste the name into the command text. Turn `{file}` into a quoted `$ROBIN_FILE` reference and pass the path in the environment. Quoting the name with `shlex.quote` is not enough: inside a longer double-quoted string such as `"{file}.bai"` it can still run. Templates that start a second shell (`sh -c "… {file}"`, `ssh host "…"`) re-parse the name there; document that.
- **Status:** fixed in our fork (`4f26cf0`, `75485ce`). A quoted `{file}` inside `$(...)` is quoted for that inner context and stays one word (`ab5151b`). Still open in the original ROBIN.

---

9. Packaging, dependencies, tests

PKG-1 HIGH · **Confirmed**: Builds aren't reproducible.

- **Where:** `pyproject.toml:38-40, 48, 55`, `target_analysis.py:3274, 3581`
- **What's wrong:**
  - `sturgeon@…@testing` tracks a moving branch.
  - `methylartist` has no git ref at all.
  - torch, polars, pyranges and ray are unpinned, and there is no lockfile.
  - ClairS-TO is pulled as `:latest`.
  - The GUI's CrossNN version lookup uses a path that doesn't exist, so it always shows "unknown".
- **Effect:** a report can't say which classifier or caller code produced it.

PKG-2 MEDIUM · **Confirmed**: There are no tests and no CI for code on `main`.

- **What's wrong:**
  - pytest's `tests/` folder holds only the two tests added after this review (`tests/test_temp_utilities.py`).
  - CI only builds the docs, on Python 3.11.
  - The mypy and black configs still target Python 3.8.

PKG-3 MEDIUM · **Confirmed**: Python 3.13+ installs fail.

- **Where:** `pyproject.toml:21`
- **What's wrong:** `requires-python = ">=3.12"`, but numpy 1.26, pandas 2.1.4 and pyarrow 16.1 have wheels only up to 3.12, and sturgeon@testing requires ≤3.12.
- **Fix:** add `<3.13`.
- **Status:** fixed in our fork (`43db1df`). Still open in the original ROBIN.

PKG-4 MEDIUM · **Confirmed**: Unpinned polars and pyranges are already causing problems.

- **What's wrong:**
  - `pl.StringCache` gives a deprecation warning on polars 2.0 (`temp_utilities.py:283`).
  - `ncls` is imported but not declared (`fusion_work.py:43`).
  - PyRanges objects are pickled into the sample folders.

PKG-5 MEDIUM · **Confirmed**: Resource files and the gitignore need cleaning up.

- **What's wrong:**
  - About 113 MB of unused or duplicate resources ship in every wheel. Examples: a 45 MB gencode GFF, two 20.6 MB MNP-Flex BED variants, and `None_data.csv.gz`. Some are byte-identical copies.

- `.gitignore` misses the v2 model files (340 MB).
- `src/robin/analysis/Untitled` is tracked in git.

> **PKG-6** LOW · **Confirmed**: The dependency list and licence need tidying.

- **What's wrong:**
  - 10 dependencies are listed twice, about 13 are never imported, and 6 that are imported aren't declared.
  - `robin.yml` leaves numpy unpinned while pip pins `<2`.
  - The package is labelled MIT but bundles GPL-3.0 nanoDx. That one is a licence question to raise.
- **Status:** partly fixed in our fork (`43db1df`): `robin.yml` pins numpy, pandas and scikit-learn, and the duplicate scikit-learn line is gone. The other duplicates, the undeclared imports and the licence question are still open.

---

10. Architecture

- **ARCH-1: Two workflow engines that have drifted apart.**
  - `workflow_ray.py` (5.2k lines) and `workflow_simple.py` each define their own `Job`, `WorkflowContext`, batcher and watcher.
  - Their `BATCH_CONFIG` differs (CNV 20 vs 50).
  - Inside workflow_ray there are three dispatch paths, each with different error detection.
  - Per-sample state is spread over about 15 dicts.
  - JOB-2, JOB-8 and ERR-1(5) all come from this.
- **ARCH-2: God files and god functions.**
  - `gui_launcher.py` is 9.2k lines.
  - `add_coverage_section` is one 5,181-line function, `add_cnv_section` is 2,260 lines and `run_snp_analysis` is 1,233 lines.
  - `fusion_work.py` is 6.5k lines.
  - None of these can be unit-tested.
- **ARCH-3: The same logic is copied into the analysis, GUI and report, and the copies have drifted.** Examples: cytoband CNV calls, fusion clustering and validation, pathogenicity checks, confidence and coverage thresholds, MGMT site parsing. REP-1, REP-2, GUI-7, GUI-8 and METH-10 are the results.
- **ARCH-4: Lots of dead code that still gets edited.** Examples:
  - matkit's copy of `merge_modkit_files`
  - about 1,000 lines in `fusion_work`
  - about 980 lines under `if False:` in `coverage.py`
  - about 800 lines in `gui_launcher`
  - 7 of 9 functions in `reporting/plotting.py`
  - `reporting/utils.py`, which is shadowed by `reporting/utils/`
  - `SQLiteStateTracker`
  - `DISABLE_MEMORY_MANAGER = True`, which is hard-coded
  - five separate model-download implementations
- **ARCH-5: Layering runs the wrong way.**
  - The analysis code imports the GUI.
  - The workflow raises "GUI not available", so headless runs still need NiceGUI.
  - `import robin` loads torch, onnxruntime, sturgeon, pysam and the rest, for every CLI call and every Ray worker.
- **ARCH-6: File contracts are undocumented.**

- Counter files are shared across modules (fusion and target use `cnv_analysis_counter.txt`, which is the cause of FUS-2).
- Many writes are non-atomic, and pickles are used everywhere.
- Coordinates, percent vs fraction, strand convention and which columns carry coverage are not written down anywhere.

- **ARCH-7: The GUI orchestrates the pipeline by polling files.** For example, a failed SNP job makes a bulk SNP queue wait up to 24 hours per sample.

---

11. Performance

ID	Problem	Measured / estimated	Where
PERF-1	Fusion GUI refresh runs an O(n^3) validation on the event loop every 30 s, and nothing reads the result	400 reads per gene pair took 21.5 s; 2,000 reads didn't finish in 10 min. The GUI freezes for all users	`gui/components/fusion.py:2736-2915, 1275-1487`
PERF-2	The MGMT timer runs methylartist inside the GUI process, on the event loop	Freezes on every MGMT batch	`gui/components/mgmt.py:550-603`
PERF-3	IGV code `time.sleep`-polls JavaScript calls that are never awaited	At least 5 s freeze on every details page load; status text is wrong	`gui/components/coverage.py:593-640, 744-1010`
PERF-4	The background scan holds a lock over all file I/O; a `ray.get(timeout=30)` runs inside an async handler	Multi-second GUI stalls with many samples	`gui_launcher.py:795-1092, 1384-1436`, `coverage.py:1639-1697`
PERF-5	The module-level `@ui.refreshable` rebuilds every client's summary	N^2 rebuilds: 5 tabs mean 75 rebuilds per 30 s	`gui/components/summary.py:64, 137, 241`
PERF-6	`stats()` waits behind busy pools	GUI stats freeze while a 30-60 min Clair3 job runs	`workflow_ray.py:3497-3498, 1309-1311, 3060-3066`
PERF-7	Each fusion batch rescans the whole-sample state and re-runs DBSCAN	O(N^2) over a run	`fusion_work.py:2675-2803, 4504-4559`
PERF-8	The per-BAM merge decodes binary columns row by row in Python, and round-trips through pandas and PyRanges	5.6 s of 11 s per 4M-row file; a vectorised `cast(pl.Utf8)` takes 0.04 s	`temp_utilities.py:223-266`, `matkit.py:992-1019`
PERF-9	BAMs are read many times (CNV 2-3×, target about 5×); ClinVar × regions is a pure-Python loop	CPU backlog at PromethION throughput	`cnv_analysis.py`, `target_analysis.py`, `lightweight_gene_analysis.py:560-612`
PERF-10	The report renders 24 per-chromosome figures at 300 dpi, then throws most away	Slow reports	`reporting/sections/cnv.py:988-1011`, `plotting.py`
PERF-11	MNP-Flex re-sends three base64 PNGs every 30 s	Constant traffic and flicker	`gui/components/mnpflex.py:722-765`

Suggested order of work

Make each step below its own change, with its own regression test. The scripts in `2026-10-16-repro/` are the starting point for those tests.

1. **Security first** *(done in our fork, with the version pins)*:
   - SEC-1: Ray dashboard on 127.0.0.1, and opt-in
   - SEC-8: MGMT command as an argument list, with no shell
   - SEC-9: pass watched file names to command templates as data

2. **Sample identity and truthful variant summaries, together** *(done in our fork)*:
   - VAR-9: only look inside the sample's own folder
   - VAR-8: count a variant only when reads support it
   - VAR-10: analyse every interval, not only the first per name
3. **Quick wins (each a few lines):** *(done in our fork; JOB-6's last two parts with step 8, batch 10b)*
   - GUI-1: guard both loops in `_refresh_classification`
   - REP-1: the RF config key
   - JOB-6: change 60 to 3600, and make it configurable
   - GUI-4: stop reading the counter file
   - CNV-5: drop the last change point
   - GUI-2: use `on_delete`
   - VAR-3: force-rebuild the tabix index
   - VAR-7: keep all panel regions
4. **CNV-1:** build the control profile without patient reads (see CNV-1; a deepcopy alone is not enough). *(done in our fork, with CNV-3)*
5. **The methylation maths, as one change with a test against `modkit pileup --combine-strands`:** *(done in our fork, except METH-7)*
   - METH-1 and METH-2: aggregate always, and recompute % from counts
   - METH-5: semi-join, and dedupe the panel
   - METH-6: drop the `Start-1`
   - METH-3: matkit denominator
   - METH-4: combine strands
   - METH-7: pass coverage through
   - METH-13, METH-14: honour MM skip modes, and decide the QS filter per read before counting. Add MM-mode and primary/supplementary test files.
6. **Target input and coverage:** *(done in our fork)*
   - TGT-1: keep every BAM's panel reads when it arrives
   - TGT-2: pass the panel explicitly to the master BED
   - TGT-4: one locked read-update-write step
   - TGT-3: depth from aligned bases. Test that the same reads split into different BAMs give the same number.
   - TGT-5: count primary records only
7. **Make failures visible:** *(done in our fork)*
   - ERR-1: failures counted, and forwarded to the GUI
   - METH-8: classifiers raise instead of logging
   - VAR-4, REP-6: "not available" instead of a negative result
8. **Scheduling state:** *(JOB-1, JOB-2, JOB-4, JOB-5, JOB-7 and CNV-2 done in our fork, with the rest of JOB-6)*
   - JOB-1: per-sample dirty flags *(done)*
   - JOB-2: a persistent file ledger *(done, with JOB-8 and JOB-4's partial-file check)*
   - CNV-2: no worker caches, and atomic writes *(done)*
   - JOB-4 *(done)*, JOB-5 *(done)*, JOB-7 *(done)* (JOB-3 was fixed with step 7)
   - Found while fixing JOB-1: on the `high` preset the Coordinator answers nothing while a job runs. A status call waited the job's full 4 s (0.01 s on `standard`), so submissions and the GUI's status calls wait for running jobs. `high` is opt-in. The cause is not found yet.
   - Found by the external review of JOB-5, older than it: a classifier run started when bed_conversion ends never adds to the running counts per queue and type, but its end takes one off, which loosens the `slow` queue's limit. And jobs that a finished job triggers still wait for room in the backlog inside its finish step;

on `high`, where completions are handled one at a time, that may hold up every other completion (not measured).
    - Found while fixing JOB-6: when the workflow's job types are known and their outputs exist, the GUI's background scan marks a timed-out sample Complete without starting a finalize, so its batch BAMs wait for the next SNP run (which now merges first) or a manual finalize. Which of the GUI's scans sees the change first in a real run is not measured. And with new reads after an SNP run, the "SNP complete" poller waits out its 24 h and then says "still running", though the run has ended.
    - Found while fixing CNV-2: `process_single_bam` has no caller in `src/` (the CNV handler always batches), and `SERIALIZE_BY_TYPE_PER_SAMPLE` is empty, so its code in the Coordinator has never run. Both are candidates for ARCH-4's dead-code removal.
    - Found while mapping JOB-2's counting steps (read in code, not measured):
        - **MGMT:** it adds a BAM's MGMT reads to `mgmt.bam` by copying a merged file over it in place (`mgmt_analysis.py:1325-1328`). No lock is used anywhere in `mgmt_analysis.py`, so two MGMT jobs for one sample at once could lose one's reads.
        - **Methylation:**
            - The merge's lock carries on without locking when `flock` fails (`temp_utilities.py:54-57`).
            - `bam_file_count` in `<sample>_metadata.json` counts merges, not BAMs: each merge adds 1 (`bed_conversion.py:319`). It is also read before the lock (`temp_utilities.py:285`; the lock is taken at `:382`).
        - **Fusion:** its accumulation has no lock (already noted under REP-6).
        - **Target coverage:** its commit was not one transaction (TGT-3) *(done in batch 10e)*.
    - Found while fixing batch 10e (read in code, not measured): when the GUI's SNP button can't hand its job to the workflow, it runs the SNP handler itself with a job whose `add_error` does nothing, so an SNP run that fails there shows no failure. The variant check's own button had the same gap, fixed in batch 10e.
9. **Security hardening:** SEC-2 to SEC-7.
10. **Pin and record versions** (PKG-1), **add CI with a test suite** (PKG-2), then start on the architecture items. Delete dead code first (ARCH-4), because it is cheap and stops fixes from landing in the wrong copy.
11. **Later, a feature rather than a fix:** per-base depth for the panel (a D4 file from `mosdepth`), to report the share of each gene's bases at ≥20× and to find low-coverage holes in panel genes. Today "above 30×/20×/10×" counts whole targets by their average depth.

Already done: the original "Error processing file … ['Chromosome', 'Start']" log noise is fixed on `main` (commit `d02add2`), with a test. Commit `5be9e04` adds a traceback and safe diagnostics to that log line, because one clinical server still logs it after the fix. Step 1 (SEC-1, SEC-8, SEC-9), the version pins, step 2 (VAR-9, VAR-8, VAR-10) and step 3's quick wins (GUI-1, REP-1, JOB-6, GUI-4, CNV-5, GUI-2, VAR-3, VAR-7) are done too; see "This fork and the original ROBIN". JOB-6's last two parts, the merge-skip and the batch-BAM deletion, were done with step 8 (the deletion already by TGT-1). The follow-ups batch 3's own review asked for soon are done as well, and so are the five findings of the 2026-10-31 review of the fix series. Step 4 (CNV-1) is done, with CNV-3, which sits in the same loop. Step 5 (the methylation maths) is done except METH-7, with a test against modkit. Step 6 (target input and coverage, TGT-1 to TGT-5) is done, with a test against samtools. Step 7 (making failures visible: ERR-1, METH-8, VAR-4, REP-6) is done, with each failure injected on purpose through a real Coordinator; of METH-9 only the reporting is done, and the classification panel doesn't yet mark its last result as old. Step 9 is done: VAR-1 and VAR-2 were checked against ClinVar's own file, and VAR-2 counts ClinVar's oncogenicity and somatic-impact labels as well as the germline one. Of step 8, JOB-1 and JOB-5 are done: no request for a classifier or SNP run is thrown away. JOB-7 is done too: a finalize request merges once, and SNP calls follow every read. So is CNV-2: a sample's CNV counts keep every batch, whichever worker added it. And JOB-2, with JOB-8: a ledger takes each BAM in once, and after a restart runs only the steps it never started. And JOB-4: the watcher sends every BAM it takes in, or says which it couldn't. Two items batches 8 and 9 noted for later are done as well: the target step shows each BAM whose coverage it could not add, and the variant check names the regions it could not check. And TGT-3's last point: a batch's coverage reaches every file, or none.